Separating Nondeterministic Time from Deterministic Space

Charlie Cruz
cjh16@rice.edu

Abstract

We show that every nondeterministic multitape Turing machine running in time $t(n) \geq n$ can be simulated deterministically in space $O(t/\sqrt{\log t})$. By the space hierarchy theorem, $\mathsf{NTIME}[n] \subsetneq \mathsf{SPACE}[n]$. In 1975, Hopcroft, Paul and Valiant simulated deterministic time t in space $O(t/\log t)$ by recomputing intermediate configurations rather than storing them. For nondeterministic machines this fails, because a second run may make different choices. We instead recompute counts of computations, which are values of sparse networks. We compute them with the grid recursion behind Anthropic's refutation of the Exponential Time Hypothesis, using our new algebraic decomposition built from walks on the Boolean cube, and evaluate the resulting integer formula depth first, one bit at a time. Since every bit of the count is available, the same bound holds for unbounded-error probabilistic time, and $\mathsf{PPTIME}[n] \subsetneq \mathsf{SPACE}[n]$. Pushing the method to space $t/\log t$ would count the satisfying assignments of 3-CNF formulas faster than any known algorithm. The proofs are formally verified in Lean.

1 Introduction

A nondeterministic machine running in time t can be simulated deterministically in space t by trying its choice sequences one after another and reusing the space. For deterministic machines one can do much better. Hopcroft, Paul and Valiant simulated time t in space $O(t/\log t)$ [HPV77], and Williams, using the tree evaluation algorithm of Cook and Mertz [CM24], improved this to $O(\sqrt{t \log t}\,)$ [Wil25]. Both simulations save space the same way, by recomputing intermediate results instead of storing them.

For nondeterministic machines, recomputation raises a consistency problem. As Hopcroft, Paul and Valiant observed, a computation that is run again may make different choices, and then the two runs need not fit together [HPV75, §2]. They asked whether $\mathsf{NTIME}[t \log t] \subseteq \mathsf{NSPACE}[t]$, and Santhanam later studied whether $\mathsf{NTIME}[t] = \mathsf{NSPACE}[t]$ for constructible t [San01].

Paterson simulated nondeterministic time $t \geq n^2$ in deterministic space $O(\sqrt{t})$ for single-tape machines [Pat72]. For multitape machines, we know of no $o(t)$-space simulation of nondeterministic time t that predates the recent refutations of the Exponential Time Hypothesis. Earlier lower bounds against small space restrict time and space together, as in $\mathsf{NTIME}[n] \not\subseteq \mathsf{DTISP}[n^c, n^{o(1)}]$ for some $c > 1$ [vM06]. We show that the algebraic decompositions in those refutations give deterministic space $O(t/\sqrt{\log t})$.

Theorem 1.1. For every function $t(n) \geq n$,

$$\mathsf{NTIME}[t] \subseteq \mathsf{SPACE}[t/\sqrt{\log t}\,].$$

The simulating machine runs in time $2^{O(t/\sqrt{\log t})}$, and t need not be constructible.

Here, as throughout, machines are multitape Turing machines and SPACE is deterministic space. The space hierarchy theorem turns Theorem 1.1 into separations.

Corollary 1.2.

(i) $\mathsf{NTIME}[n] \subsetneq \mathsf{SPACE}[n]$. More generally, $\mathsf{NTIME}[t] \subsetneq \mathsf{SPACE}[t]$ and $\mathsf{NTIME}[t] \subsetneq \mathsf{NSPACE}[t]$ for every space-constructible $t(n) \geq n$.

(ii) $\mathsf{NTIME}[t\sqrt{\log t}\,] \subseteq \mathsf{SPACE}[t]$ for every $t(n) \geq n$.

(iii) There is a language $\mathcal{L} \in \mathsf{SPACE}[n]$ such that every nondeterministic machine deciding $\mathcal{L}$ or its complement takes, for some $c > 0$, at least $c\,n\sqrt{\log n}$ steps on some input of every large length n.

(iv) The same holds for unbounded-error probabilistic time: $\mathsf{PPTIME}[t] \subsetneq \mathsf{SPACE}[t]$ for every space-constructible $t(n) \geq n$, so $\mathsf{PPTIME}[n] \subsetneq \mathsf{SPACE}[n]$ and $\mathsf{BPTIME}[n] \subsetneq \mathsf{SPACE}[n]$, and (iii) holds for probabilistic machines that decide $\mathcal{L}$ or its complement with unbounded error.

Part (i) settles the question Santhanam studied, in the negative. With the theorem of Paul, Pippenger, Szemerédi and Trotter that $\mathsf{TIME}[n] \subsetneq \mathsf{NTIME}[n]$ [PPST83], it makes both inclusions in $\mathsf{TIME}[n] \subseteq \mathsf{NTIME}[n] \subseteq \mathsf{SPACE}[n]$ strict. Part (ii) is a step toward the question of Hopcroft, Paul and Valiant. Part (iii) is a lower bound proved by an algorithm, much as their simulation separated deterministic time from space [HPV77].

The idea is to count rather than search. Our simulation never follows the guesses of a single computation; it sums over all of them and computes the number of accepting choice strings. Suppose two groups of time intervals of the computation share some boundary data θ: states, head positions and the tape contents that pass from one group to the other. If $A(z)$ and $B(z)$ count the local choice strings on the two sides that are consistent with θ, then

$$\sum_{\theta} A(\theta)\, B(\theta)$$

counts the compatible pairs. Each factor depends on θ alone. It can be recomputed whenever it is needed, and the two sides always fit together.

This makes recomputation consistent, but does not yet save space: the boundary data θ can be as long as the computation. To shorten the sums, we use algebraic decompositions from the refutations of the Exponential Time Hypothesis [Ope26, Ant26], the conjecture that 3-SAT on n variables needs time $2^{\Omega(n)}$ [IP01]. Before these works, no algorithm was known to count the satisfying assignments of a 3-CNF formula with m clauses in time $2^{o(m)}$; their algorithms take time $2^{O(m/\sqrt{\log m})}$ [Ope26, App. D.2], [Ant26, Main Theorem].

These counting bounds do not directly give our simulation. Standard reductions encode a computation of length t by a formula of size $O(t \log t)$ [PF79, Coo88]; substituting this size into the counting bound gives no saving over brute force. The blocking of Hopcroft, Paul and Valiant gives $O(t)$ variables instead, but its constraints have arity $\Theta(\log t)$, and the network they form is too large to store. The counting algorithms bound time, not space. We need a counting algorithm that reads the network through local queries and computes each bit of its value in space $O(t/\sqrt{\log t})$; Theorem 2.4 reduces Theorem 1.1 to exactly this problem.

We obtain such an algorithm from the grid recursion of Anthropic [Ant26], using a new decomposition together with a quadratic potential that Zhuang also found [Zhu26]. Evaluating the recursion depth first over the integers then gives space about the logarithm of its running

time. The square-root saving comes from about $\log \log t$ merges, each halving this quadratic potential.

Because the simulator computes any bit of the exact number of accepting choice strings, it handles unbounded error as well. So $\mathsf{PPTIME}[t]$ and $\mathsf{BPTIME}[t]$ lie in $\mathsf{SPACE}[t/\sqrt{\log t}]$ (Theorem 7.1), which gives part (iv).

1.1 Overview of the proof

The simulation begins by turning computations into networks. Following Hopcroft, Paul and Valiant, cut time into intervals of $\beta = \lceil \log t \rceil$ steps and every tape into blocks of β cells. A head moves by at most one block per interval, so the head schedule, which records these moves, takes $O(t/\log t)$ bits. Fix one schedule. Put a Boolean variable on every bit that an interval leaves behind, and on every interval a constraint that counts the local choice strings turning what it reads into what it writes. A tape block keeps its contents between visits, so each variable is touched by at most two intervals, and the constraints form a network G with $O(t)$ edges. Its value $Z(G)$ is the number of accepting choice strings that follow the schedule. Since these counts are nonnegative, the input is accepted exactly when $Z(G) > 0$ for some schedule, and the simulator tries the schedules one at a time.

What remains is to compute the bits of $Z(G)$, for a network of size $s = O(t)$, in space $O(s/\sqrt{\log s})$. We write $Z(G)$ as an arithmetic formula over the integers whose sums and products are indexed by bit strings. Any bit of such a formula can be computed depth first, adding children one binary column at a time, in space about the total length of the indices on one root-to-leaf path (Proposition 3.2); this is the principle behind the work of Borodin [Bor77] and of Lokshtanov and Nederlof [LN10]. A brute-force formula sums over all $O(s)$ bits of the edges and needs linear space. To save space, we must shorten the indices.

We shorten them by a recursion on cliques. Following Williams's algorithm for 2-CSP [Wil05], group the vertices of the network into $q = 2^{h_*}$ nodes with h_*-bit names. Each pair of nodes now shares a string, the bits of the edges between them, and $Z(G)$ becomes a sum over these strings of a product of one table entry per node. Read the first half of each name as a row and the second half as a column of a square grid. Merging the nodes of each row leaves $2^{h_*/2}$ nodes whose names have $h_*/2$ bits, and so does merging the columns. The table of a merged node sums over the strings inside its row or column.

In a row merge, pairs inside a row vanish into the merged node, and pairs inside a column are carried along unchanged. Pairs whose endpoints differ in both row and column cause the difficulty. Two rows and two columns determine a rectangle with corners x and y in the upper row and u and v in the lower (Fig. 2). Each diagonal carries a string, with one copy held at each end. The equality tests of the two diagonals multiply to the crossing $[x = v][y = u]$. Merging the rows separates (x, y) from (u, v); merging the columns separates (x, u) from (y, v). Either way, every bit of the crossing must pass the cut.

Over a ring, the crossing comes apart. For single bits x, y, u, v, the one-bit identity reads

$$[x = v][y = u] = [x = u][y = v] - \varepsilon(x, y)\,\varepsilon(u, v), \tag{1}$$

where $\varepsilon(0, 1) = 1$, $\varepsilon(1, 0) = -1$ and ε vanishes on equal bits. At $(x, y, u, v) = (0, 1, 0, 1)$ both diagonals disagree, and the right side is $1 - 1 = 0$: the two terms cancel on this inconsistent assignment. The first term is a function of (x, u) times a function of (y, v), so no bit crosses the column cut, and in the second term no bit crosses the row cut. This use of subtraction matters: polynomially many pieces combined by OR and AND cannot achieve the same saving

(Proposition 8.1). For longer strings, multiplying (1) over the coordinates gives terms that are just as cheap, but naming one of them costs a bit per coordinate, as much as the strings themselves. Anthropic found polynomially many pieces built from operators on polynomials [Ant26, Appendix B]. We construct ours from walks on the cube.

The pieces replace the two labels of a crossing by one message. To measure the saving, we use weights that control squared code lengths. A label is a pair of multisets of requests, with type i weighing $i+1$. A string of r bits becomes a label with one request of each type below r, of weight $r(r+1)/2$, while a label of weight at most W has an $O(\sqrt{W})$-bit code. Let the diagonal labels have weights a, b, and put $H = a + b$. We write the crossing as a sum of pieces Π_k, for even grades $|k| \le 2H$; their factorizations use messages of weight at most

$$H + k/2 \quad \text{across rows,} \qquad H - k/2 \quad \text{across columns}$$

(Corollary 5.5). Splitting the grade between the two factors adds $O(\log(H+2))$ index bits; the merge records the split alongside the grade, so the new label holds only the message. Each message is one label. At grade 0 it weighs at most $a + b$, like the two labels together, but fills one field instead of two. Other grades move weight between the cuts, but the budgets always add to $2H$, just as the two original labels would weigh H at each cut.

The pieces come from a signed sum over routings, weighted by powers of a formal variable z. A shared message does not record which diagonal sent a request, so when requests are matched across the cut, corners are also joined along the sides. An identity for walks on the cube keeps the diagonal equality tests exact, while every routing that uses a side contains a factor that vanishes at $z = 1$. The coefficients of the resulting Laurent polynomial are the pieces. For the row cut, a step to a heavy intermediate type costs a high power of z, which bounds the message weight in terms of the grade; transposition exchanges the cuts and reverses the grade (Section 5).

We compare the two merges by the potential

$$\Phi = (q-1)^2 \sum_{\{u,v\}} a_{uv}$$

of the pair budgets a_{uv}. For each choice of pieces, merge all rows or all columns and pack what each pair of new nodes shares into one label. Packing scales the total weight budget by a factor L, the number of fields plus one for padding. Two rows share the labels of their $\sqrt{q}$ vertical pairs and one message per rectangle, so $L = q/2 + O(\sqrt{q})$, compared with $q + 1$ without pieces. A pair within one row or column is carried by only one merge, and a rectangle's budgets add to $2H$ across both. Together these give $\Phi_{\text{row}} + \Phi_{\text{col}} \le \Phi$ (Lemma 6.1).

Initially $\Phi = O(s^2)$ and the name length is $h = h_* = \Theta(\log s)$ (Lemma 4.3). Choosing the cheaper merge at every stage halves the potential as it halves h, so Φ/h never increases. The accounting must be exact: losing a constant factor at each of the $\log\log s$ merges would cost a power of $\log s$. At $h = 1$, the two remaining nodes share a label of weight $O(s^2/\log s)$, with a code of $O(s/\sqrt{\log s})$ bits.

The simulator never writes out the network or the clique tables. A merged table entry is evaluated from its definition each time it is needed, as in Anthropic's algorithm [Ant26, Algorithm 1]: it sums over the old labels of one row or column and evaluates the old entries one at a time. The potential controls label lengths; the remaining issue is how many indices the evaluator holds at once. A path chooses pieces through successive merges, reaches the last label, and then follows at most one table entry per name length, never returning to a clique value (Fig. 1). The bounds on the table-entry indices form a summable geometric series, and the remaining indices fit in the same $O(s/\sqrt{\log s})$ budget (Lemma 6.3).

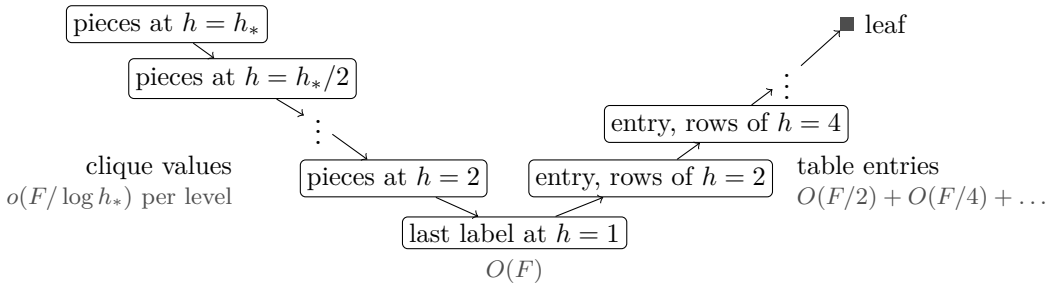


Figure 1: A root-to-leaf path. The V shape tracks the lengths of clique names: they decrease through choices of pieces and increase through table entries. Every arrow goes deeper in the formula; a table entry never calls a clique value. With $F = s/\sqrt{h_*}$, the index lengths total $O(F)$, including any coefficient recursion at the end (Lemma 6.3).

Section 2 reduces the simulation to counting. Sections 3 to 6 prove the counting theorem from scratch, using ideas from [Ant26] but none of its results. Sections 7 and 8 give the counting-class consequences and the limits of the method.

1.2 Contributions and related work

The building blocks come from classical complexity theory and from the recent refutations of the Exponential Time Hypothesis by OpenAI [Ope26] and, independently, by Anthropic [Ant26]. The tape blocking is that of Hopcroft, Paul and Valiant [HPV77]. From Anthropic we take the grid of nodes, the choice of a row or column merge for each term, and the expansion of each crossing into signed pieces graded by an integer. We arrived independently at the halving of node names and the invariant of Section 6; Zhuang's manuscript [Zhu26] develops related ideas, including a quadratic potential and pieces built from the one-bit identity (1).

Our contributions are the application to nondeterministic time, a more elementary algebra for the crossing pieces and a self-contained proof of the counting bound the simulation needs. We obtain logspace-accessible networks of size $O(t)$ from blocked computations (Section 2) and construct new crossing pieces from the cube-walk identity of Lemma 5.1. Their two factorizations follow from elementary inequalities for the cube distance and from transposition, with message-weight budgets summing to exactly twice the weight of the crossing. The simulation uses explicit multiset codes, a deterministic construction of the first clique, and direct evaluation of integer bits. These ingredients let us carry out the recursion in small space from local access to the network.

Rain Yang found a similar result while working with Toniann Pitassi on extending Williams's low-space simulation [Wil25] to nondeterministic time. Using ideas from OpenAI's refutation and the assistance of Claude Opus 5.5, Yang obtained a proof that $\mathsf{NTIME}[T]$ is contained in $\mathsf{SPACE}[T(\log\log T)^2/\sqrt{\log T}]$. Sam McGuire independently found the bound $t/\sqrt{\log t}$ of Theorem 1.1 (personal communication); for time-constructible $t \geq n$, his note [McG26] simulates nondeterministic and bounded-error probabilistic time t in deterministic time $2^{O(t/\sqrt{\log t})}$.

1.3 Formal verification

The proof in this paper is formally verified in Lean 4 [dMU21], using Mathlib [Mat20], CSLib [CSL] and Complexitylib [Cplx]. The development is available at `https://github.com/SeniorMars/`

`ntime.dspace`. It proves Theorems 1.1 and 7.1 for every $t(n) \geq n$, together with Corollaries 1.2 and 7.2, using Complexitylib's definitions of machines and classes and only Lean's standard axioms. The classes $\oplus\mathsf{TIME}$, $\mathsf{Mod}_q\mathsf{TIME}$ and $\mathsf{C}_=\mathsf{TIME}$, which Complexitylib lacks, are defined in the development as in Section 7. The development follows the paper section by section, except that the halting test and the counts of computations of Lemma 2.2 are taken as accept counts of modified machines. Propositions 8.1 and 8.2 and Remark 8.3, for the oracle used there, are verified as well, and so are the implications of Section 8. Machines and resources are those of CSLib and Complexitylib: multitape Turing machines whose binary input sits on a read-only tape, with space counted as the number of work-tape cells visited.

Comparator [Cmp] confirms that the proved statements of Theorem 1.1, Corollary 1.2 (i)–(iii) and the bounds for PPTIME and BPTIME match statements written against Complexitylib alone, and that their proofs use only the standard axioms.

2 Networks and nondeterministic time

2.1 Networks

We use multitape Turing machines with a read-only input tape and k work tapes, and we measure space by the number of work cells visited; the classes TIME, SPACE, NTIME and NSPACE are the usual ones [AB09]. A nondeterministic machine decides a language if all its computations halt and an input belongs to the language exactly when some computation on it accepts. At the cost of a constant factor in time, we may assume that a nondeterministic machine has at most two applicable transitions in each configuration and stays halted once it halts: a choice among more transitions becomes a binary tree of choices with one leaf per transition, which preserves the numbers of accepting and rejecting computations. A choice string $w \in \{0,1\}^T$ then determines a computation of length at most T, the bit w_j selecting the transition at step j whenever there are two. A function S is space-constructible if $S(n)$ can be computed from 1^n in space $O(S(n))$. Logarithms are to base 2, and $[\mathcal{P}]$ is 1 if $\mathcal{P}$ holds and 0 otherwise.

We count accepting computations as solutions of sparse constraint systems, written as networks. A *network* is a finite graph G, possibly with parallel edges but without loops, with a function $f_v \colon \{0,1\}^{\delta(v)} \to \mathbb{Z}$ at every vertex v; here $\delta(v)$, the ports of v, is the ordered list of edges at v. The value of G is

$$Z(G) = \sum_{x \in \{0,1\}^{E(G)}} \prod_{v \in V(G)} f_v(x|_{\delta(v)}).$$

The edges are Boolean variables, and the vertices are constraints on the variables they touch; a vertex without edges contributes the constant f_v. In the language of Cai, Lu and Xia, G is a signature grid over the Boolean domain and $Z(G)$ is its Holant value [CLX09].

Every constraint system in which each variable occurs in one or two constraints is a network with the same value. A variable in two constraints becomes an edge between them, and a variable in one becomes an edge to a new vertex whose function is 1. A variable in no constraint can be removed, at the price of a factor 2 in the count.

The networks that come from a computation have $O(t)$ edges, far too many to write down in sublinear space. So the algorithm only asks local questions about them.

Definition 2.1. A family of networks has *logspace access at size s* if, with constants uniform across the family:

(a) every network G in the family has a description of length $s^{O(1)}$, at most $O(s)$ edges, $s^{O(1)}$ vertices, maximum degree $O(\log s)$, and vertex functions with values in $\{0, \dots, s^{O(1)}\}$; and

(b) given read-only access to the description of G, one can compute in space $O(\log s)$ and time $s^{O(1)}$ its vertex and edge names, which are distinct elements of a universe of size $s^{O(1)}$; its incidences and port orders; and the value of any vertex function at any argument.

This is an implicit representation with logspace decoding, related to the succinct representations studied by Galperin and Wigderson [GW83] and Wagner [Wag86]. Both an argument $x|_{\delta(v)}$ of a vertex function and its value take $O(\log s)$ bits, so they fit in the workspace. And since G has $O(s)$ edges and $s^{O(1)}$ vertices with polynomially bounded values, $0 \le Z(G) < 2^{s^{O(1)}}$.

2.2 Blocked computations as networks

Once the heads' movements between blocks are fixed, the blocking of Hopcroft, Paul and Valiant turns a computation into a network. Fix a nondeterministic machine $\mathcal{M}$ with k work tapes, an input x and an integer $t \ge \max(|x|, 4)$. Put

$$\beta = \lceil \log t \rceil, \qquad J = \lceil t/\beta \rceil, \qquad \bar{t} = J\beta,$$

so that $t \le \bar{t} < 2t$. Cut the first $\bar{t}$ steps into J intervals of β steps, and every tape into blocks of β cells. A head schedule $\pi \in \{-1, 0, 1\}^{(k+1)\times J}$ records, for every head and every interval, whether the head ends the interval one block to the left of where it started, in the same block, or one block to the right. It has $O(t/\log t)$ bits, and every choice string follows exactly one schedule. The schedule plays the part of the computation graph of a block-respecting machine in the simulations of Hopcroft, Paul and Valiant and of Williams, who likewise enumerate its short encodings [HPV77, Wil25].

Besides acceptance, we will need to test whether any computation is still running at step $\bar{t}$. This lets the simulator find a sufficient clock without computing $t(n)$. We will also distinguish choice strings from computations, since the counting classes of Section 7 use both.

Lemma 2.2. For every input x, every $t \ge \max(|x|, 4)$, every schedule π and every condition $\kappa \in \{\mathrm{acc}, \mathrm{run}\}$ there is a network $G = G^{\kappa}_{x,t,\pi}$ with the following properties, with constants depending only on $\mathcal{M}$.

(a) G has $O(t)$ edges and vertices, degree $O(\log t)$, and function values in $\{0, \dots, 2^{\beta}\}$.

(b) Given x, t, κ, π, the vertices, edges, incidences and port orders of G, and the value of any function at any argument, can be computed in space $O(\log t)$ and polynomial time.

(c) $Z(G)$ is the number of choice strings $w \in \{0,1\}^{\bar{t}}$ whose computation follows π and, at step $\bar{t}$, has halted and accepted (if $\kappa = \mathrm{acc}$) or is still running (if $\kappa = \mathrm{run}$). In particular $\sum_{\pi} Z(G^{\kappa}_{x,t,\pi}) \le 2^{\bar{t}}$.

(d) There are also networks $\tilde{G} = \tilde{G}^{\kappa}_{x,t,\pi}$ satisfying (a) and (b) such that $Z(\tilde{G})$ is the number of computations, rather than of choice strings, that follow π and satisfy κ at step $\bar{t}$.

The main vertices of G are the J intervals. The function at interval j counts the local choice strings in $\{0,1\}^{\beta}$ that turn the tape contents and control data it reads into those it writes, so its values can reach 2^{β}. A block visited by intervals $j_1 < j_2 < j_3$ gives a chain of edges, from j_1 to j_2 and from j_2 to j_3, rather than a vertex of high degree.

Proof of Lemma 2.2. In β steps a head moves by at most β cells. On each tape it therefore stays within the window of at most three consecutive blocks centered on the block it occupies at the start of the interval, and the schedule π determines this window for every interval.

The entering data of an interval consist of the machine state, the offsets of all $k+1$ heads within their starting blocks, and the contents of the window of each work head at the start of the interval. The leaving data consist of the state, the head offsets within the ending blocks prescribed by π, and the contents of the same work-tape windows at the end of the interval. Both take $O(\beta)$ bits.

For entering data η and leaving data η' of interval j, let $\tilde{f}_j(\eta, \eta')$ be the number of local choice strings $u \in \{0,1\}^\beta$ for which running $\mathcal{M}$ for β steps from η with choices u produces η' and moves every head into the block prescribed by π; in the last interval $j = J$ we also require the configuration at step $\bar{t}$ to satisfy κ. Then $0 \le \tilde{f}_j(\eta, \eta') \le 2^\beta$. Given η and η', one computes $\tilde{f}_j(\eta, \eta')$ in space $O(\log t)$ and polynomial time by replaying β steps on $O(\beta)$ work cells for each $u \in \{0,1\}^\beta$, reading the input tape at the positions determined by π.

Take a Boolean variable for each bit of the leaving data of each interval $j \in [J]$. The entering control data (state and head offsets) of interval j are the leaving control data of interval $j-1$, or the initial state and zero offsets when $j = 1$. The entering contents of each block in interval j's window are the leaving contents of that block from the last earlier interval whose window contained it, or the initial blank contents if there is none. So every variable occurs in the constraint $\tilde{f}_j$ of the interval that writes it and in at most one later constraint $\tilde{f}_{j'}$, that of the next interval whose window contains the block. Substituting the known initial constants into $\tilde{f}_1, \dots, \tilde{f}_J$ yields a constraint system with $O(J\beta) = O(t)$ variables, each in one or two constraints, and J constraints of arity $O(\beta)$, which we convert to a network G as in Section 2.1. This proves (a). Part (b) holds because the interval that last wrote any entering block can be found by scanning π backward with counters of $O(\log t)$ bits.

For (c), write each $\tilde{f}_j$ as a sum of indicators over $u_j \in \{0,1\}^\beta$. Each term of the expanded sum for $Z(G)$ specifies a full choice string $w = u_1 \cdots u_J \in \{0,1\}^t$ together with an assignment to all leaving-data variables. By induction on $j = 1, \dots, J$, the product of the J indicators for a given pair $(w, \text{assignment})$ is 1 if the assignment equals the true leaving data of the computation of w after each interval and that computation follows π and satisfies κ, and 0 otherwise. Summing over the assignments and the choice strings gives (c), and the last claim holds because every choice string follows exactly one schedule.

For (d), modify $\tilde{f}_j$ so that it counts only local choice strings $u \in \{0,1\}^\beta$ whose bit is 0 at every step where at most one transition applies, in particular at every step after the machine has halted. Computations correspond bijectively to such choice strings, which gives the networks $\tilde{G}^\kappa_{x,t,\pi}$. □

2.3 From counting to simulation

The blocking lemma leaves a counting problem with local access. A machine computes an integer Z in space S if, given the binary index of a bit position, it outputs that bit of Z in space S. We prove the following bound in Sections 3 to 6.

Theorem 2.3. Let a family of networks have logspace access at size s. Given s and the description of a network G in the family, $Z(G)$ can be computed in space $O(s/\sqrt{\log s})$ and time $2^{O(s/\sqrt{\log s})}$. The constants depend only on the family.

Given an algorithm for the bits of each network value, the simulator can enumerate schedules

and test acceptance. This step adds $O(t/\log t)$ space for the schedule, independently of how the values are computed. We state it for a general counting bound $D(s)$ so that this overhead remains explicit.

Theorem 2.4. Let $D(s) \geq \log s$ be nondecreasing with $D(2s) = O(D(s))$. Suppose that the conclusion of Theorem 2.3 holds with space $O(D(s))$ and time $2^{O(D(s))}$ for every family with logspace access at size s. Then for every $t(n) \geq n$, every language in $\mathsf{NTIME}[t]$ is decided by a deterministic machine in space $O(D(t) + t/\log t)$ and time $2^{O(D(t)+t/\log t)}$.

Proof. Let $\mathcal{M}$ decide a language in $\mathsf{NTIME}[t]$ with running time at most $c_{\mathcal{M}}t(n)$. For a candidate clock $u \geq \max(|x|, 4)$ and a condition $\kappa \in \{\mathrm{acc}, \mathrm{run}\}$, the networks $G^{\kappa}_{x,u,\pi}$ form a family with logspace access at size $s = c'_{\mathcal{M}}u$, for a constant $c'_{\mathcal{M}}$ depending only on $\mathcal{M}$ (Lemma 2.2(a,b)). By Lemma 2.2(c), and since all these values are nonnegative, some computation is accepting, respectively still running, at step $\bar{u}$ exactly when $Z(G^{\kappa}_{x,u,\pi}) \neq 0$ for some schedule π.

The simulator tests this by going through the strings π of $2(k+1)J = O(u/\log u)$ bits, two bits for each entry of a schedule, and skipping strings that encode no schedule. For each remaining string, it goes through the bit positions $0, \ldots, 2u-1$ of $Z(G^{\kappa}_{x,u,\pi})$. These positions suffice, because $Z(G^{\kappa}_{x,u,\pi}) \leq 2^{\bar{u}} < 2^{2u}$. The current string and position take $O(u/\log u)$ bits, and each bit of a network value is computed in space $O(D(c'_{\mathcal{M}}u)) = O(D(u))$, reusing the space. The test takes time $2^{O(D(u)+u/\log u)}$.

Since $t(n)$ need not be constructible, the simulator uses the usual doubling. It tries the clocks $u = u_0, 2u_0, 4u_0, \ldots$ in turn, starting from $u_0 = \max(|x|, 4)$ and reusing space, and at each clock it first tests whether some computation is still running at step $\bar{u}$. As soon as none is, every computation has halted within $\bar{u}$ steps, and the simulator accepts if and only if some computation accepts at step $\bar{u}$. Because $\mathcal{M}$ halts within $c_{\mathcal{M}}t(|x|)$ steps on all branches, the loop stops at a clock $u < 2\max(c_{\mathcal{M}}t(|x|), u_0)$, and $D(2s) = O(D(s))$ gives the stated space and time bounds. □

Proof of Theorem 1.1. Theorem 2.3 supplies the hypothesis of Theorem 2.4 with $D(s) = s/\sqrt{\log s}$. □

The separations use the space hierarchy theorem in an almost-everywhere form, whose standard proof we recall in Appendix B. Only Corollary 1.2(iii) needs this form.

Lemma 2.5 ([SHL65])**.** Let $S(n) \geq n$ be space-constructible. There is a language $\mathcal{L}_S \in \mathsf{SPACE}[S]$ such that for every deterministic machine deciding $\mathcal{L}_S$ there is $c > 0$ such that, for all sufficiently large n, the machine visits more than $c\,S(n)$ work cells on some input of length n.

Proof of Corollary 1.2. (i) Let $\mathcal{L}_t$ be the language of Lemma 2.5 for $S = t$. By Theorem 1.1, every language in $\mathsf{NTIME}[t]$ is decided in space $O(t/\sqrt{\log t})$, which is less than $c\,t(n)$ for every $c > 0$ and all large n. Thus $\mathcal{L}_t \notin \mathsf{NTIME}[t]$, while $\mathsf{NTIME}[t] \subseteq \mathsf{SPACE}[t]$. Since $\mathsf{NTIME}[t] \subseteq \mathsf{NSPACE}[t]$ and $\mathcal{L}_t \in \mathsf{SPACE}[t] \subseteq \mathsf{NSPACE}[t]$, we also have $\mathsf{NTIME}[t] \subsetneq \mathsf{NSPACE}[t]$. (ii) If $T = t\sqrt{\log t}$, then $\log T \geq \log t$, so $T/\sqrt{\log T} \leq t$. (iii) The argument of (i), applied to one nondeterministic machine, is in Appendix A. (iv) The same arguments apply with Theorem 7.1 in place of Theorem 1.1, since it holds for every $t(n) \geq n$ as well. □

3 Bitwise evaluation of integer formulas

We now prove the counting theorem, starting with bitwise evaluation of implicit integer formulas. Their sums have exponential fan-in, like Valiant's exponential sums [Val79]. The evaluator needs

space both to keep its place along one root-to-leaf path and to recover the node at that place. The following definition accounts for these separately: the ℓ-cost measures the indices and arithmetic workspace along a path, and S-explicitness bounds the space needed to inspect a node from its address.

Definition 3.1. A *formula* is a finite rooted tree in which every inner node is a sum node or a product node with an index length $w \geq 0$; such a node has fan-in 2^w, one child for each index in $\{0,1\}^w$, and its value is the sum, respectively the product, of the values of its children. Every leaf carries an integer. A node is named by its *address*, the sequence of indices on the path from the root to it, each written in binary and preceded by its length in a self-delimiting code, so that indices of length 0 are visible.

For $\ell \geq 1$, charge each sum node of index length w the amount $w + \ell$ and each product node of index length w the amount $\ell(w+1)$; the *ℓ-cost* of a formula is the largest total charge of the inner nodes on a path from the root to a leaf.

A formula given by an input is *S-explicit* if a deterministic machine that always halts, reading the input and an address, computes in space S the kind and index length of the node there and, at a leaf, its value in binary; this is a uniformity condition in the spirit of Ruzzo's direct connection language [Ruz81].

The explicit machine keeps no state between queries, so the evaluator can reuse its S cells.

Brute force is already a formula of this kind. For a network G with M edges, take a sum node of index length M, with one child for each edge assignment $x \in \{0,1\}^M$. Each child is a product node of index length $\lceil \log |V(G)| \rceil$, with the leaf $f_v(x|_{\delta(v)})$ at the index for vertex v and the leaf 1 at indices that name no vertex. This formula has value $Z(G)$. Its ℓ-cost is $M + O(\ell \log s)$, and with logspace access it is $O(\log s)$-explicit.

The evaluation computes bits, and values may be negative. Bit j of an integer V means bit j of the least nonnegative residue of V modulo 2^{j+1}, so that a negative integer has its bits in two's complement.

Proposition 3.2. Let $\ell \geq 1$, and let a formula of ℓ-cost D, with index lengths below 2^ℓ, be S-explicit for an input of length n, where $S \geq \log(n+2)$ and every leaf value has absolute value below 2^S. Given $j < 2^\ell$, bit j of its value can be computed in space $B = O(D + S)$ and time $2^{O(B)}$.

Proof. First replace every product node of index length w by a balanced binary tree of depth w, whose left and right choices are the bits of the old index. For a binary string π of length at most w, let P_π be the product of the children whose index begins with π, so that $P_\pi = P_{\pi 0} P_{\pi 1}$ for $|\pi| < w$, and P_π is the child π for $|\pi| = w$. A product node of index length w thus becomes w levels of binary products, which its charge $\ell(w+1)$ pays for, and the strings π become part of the address.

For a node u, write $b_i(u)$ for bit i of its value. The evaluator computes $b_j(u)$ recursively and depth first, by the schoolbook rules, as in Borodin's evaluation of shallow circuits in small space [Bor77]. At an inner node it scans the columns $i = 0, 1, \ldots, j$, starting from the carry $\gamma_0 = 0$, and puts

$$b_i(u) = (\gamma_i + A_i) \bmod 2, \qquad \gamma_{i+1} = \left\lfloor \frac{\gamma_i + A_i}{2} \right\rfloor,$$

where the column total is

$$A_i = \sum_\theta b_i(\text{child } \theta) \quad \text{at a sum node,} \qquad A_i = \sum_{\alpha=0}^{i} b_\alpha(P_{\pi 0})\, b_{i-\alpha}(P_{\pi 1}) \quad \text{at a binary product.}$$

Each bit of a child comes from a recursive call, whose space is released afterward. At a leaf the evaluator asks the explicit machine for the value; a negative leaf $-m$ has the complements of the bits of $m-1$. These are the rules for adding and multiplying modulo 2^{j+1}, so by induction the evaluator computes $b_j(u)$ correctly, for values of either sign.

At a sum node with 2^w children the carry stays at most 2^w, and at a binary product it stays at most $j+1$. Since $j < 2^\ell$, the frame of a sum node, holding i, the carry and the running total, takes $O(w+\ell)$ bits beyond its index, and the frame of a binary product node $O(\ell)$ bits. At any moment the stack holds one frame for each node on the path from the root to the current node, and the address holds the indices on the path, each with its length in $O(\ell)$ bits. Together they take $O(D)$ bits, and the explicit machine reuses its S cells from call to call. The recursion has bounded depth, so the evaluator halts and never repeats a configuration. It has at most $n\,2^{O(B)}$ configurations, and $S \geq \log n$ gives the time bound. □

4 Labels and cliques

The evaluator's space bound depends on the lengths of its indices. To shorten them, we will group the network into cliques and repeatedly merge their nodes. Each merge must leave labels that can be used in the next merge, with budgets that control their code lengths. We define these labels, prove the coding bound and then build the first clique with potential $O(s^2)$.

4.1 Labels and their codes

A label is what two nodes share: initially a string of bits, and after a merge the information shared by two rows or columns.

A request has a type $i \in \mathbb{N} = \{0, 1, 2, \dots\}$ and the weight $i+1$. A multiset σ of requests has a multiplicity $\mu_\sigma(i) \in \mathbb{N}$ for every type, almost all zero, and its weight is

$$\operatorname{wt}\sigma = \sum_i \mu_\sigma(i)\,(i+1).$$

A label is a pair $\lambda = (\sigma_1, \sigma_2)$ of multisets of requests, with weight $\operatorname{wt}\lambda = \operatorname{wt}\sigma_1 + \operatorname{wt}\sigma_2$. Let $\Lambda(a)$ be the set of labels of weight exactly a, and $\Lambda_{\leq}(a)$ the set of labels of weight at most a.

A string of b bits becomes a label with one request of each type $0, \dots, b-1$ (Section 4.3), of weight $b(b+1)/2$. Thus its weight grows quadratically in b, while its code stays $O(b)$ bits long.

In a clique (Section 4.2) a label sits on a pair of nodes $u < v$, compared by their names in lexicographic order. Then σ_1 holds the requests that u sends to v, and σ_2 those that v sends to u. Each endpoint sees the label as a pair (out, in) of the requests it sends and receives: u sees (σ_1, σ_2), and v sees (σ_2, σ_1).

Lemma 4.1. Put $w(0) = 0$ and $w(a) = \lfloor 28\sqrt{a} \rfloor + 2$ for $a \geq 1$, so that $w(a) \leq 30\sqrt{a}$ for all $a \geq 0$. For every $a \geq 0$ there is an injective map from $\Lambda_{\leq}(a)$ into $\{0,1\}^{w(a)}$. Given a and read access to a string, one can decide whether it is the code of a label and compute, for a code, the multiplicity of any type in either multiset and the weight, in space $O(\log(a+2))$.

Proof. The only label of weight 0 is the empty one, and its code is the empty string. Let σ be a multiset of requests of weight W, let $p_1 < \cdots < p_m$ be the distinct weights of its requests, and let $\mu_\nu \geq 1$ be the number of requests of weight p_ν. Encode σ by the Elias gamma codes [Eli75] of $m+1$ and, for $\nu = 1, \dots, m$, of the gap $g_\nu = p_\nu - p_{\nu-1}$ (with $p_0 = 0$) and of μ_ν. The gamma

code of $x \geq 1$ has $2\lfloor \log x \rfloor + 1$ bits, and a concatenation of gamma codes can be parsed from the left. The p_ν are distinct positive integers with sum at most W, so $m \leq \sqrt{2W}$, and the code of $m+1$ has at most $2m+1$ bits.

For $j \geq 1$ let r_j be the number of gaps of length at least 2^j and r'_j the number of multiplicities at least 2^j. The gap g_ν is a summand of each of the $m - \nu + 1$ numbers $p_\nu, \dots, p_m$, so $W \geq \sum_\nu p_\nu \geq 2^j(1 + 2 + \cdots + r_j) \geq 2^{j-1} r_j^2$. The weights with multiplicity at least 2^j are distinct, so $W \geq 2^j(1 + 2 + \cdots + r'_j) \geq 2^{j-1} r_j'^2$. Hence $r_j, r'_j \leq \sqrt{2W}\, 2^{-j/2}$, and the gaps and multiplicities take

$$\sum_\nu \bigl(2\lfloor \log g_\nu \rfloor + 2\lfloor \log \mu_\nu \rfloor + 2\bigr) = 2m + 2\sum_{j \geq 1} (r_j + r'_j) \leq 2m + \frac{4}{\sqrt{2}-1}\sqrt{2W}$$

bits. In all, σ takes at most $(4 + \frac{4}{\sqrt{2}-1})\sqrt{2W} + 1 \leq 19.4\sqrt{W} + 1$ bits.

For $a \geq 1$, encode a label $(\sigma_1, \sigma_2) \in \Lambda_{\leq}(a)$ by the codes of σ_1 and σ_2, followed by zeros up to length $w(a)$. Writing $a_i = \operatorname{wt} \sigma_i$, we have $\sqrt{a_1} + \sqrt{a_2} \leq \sqrt{2a}$ since $a_1 + a_2 \leq a$, so the two codes take at most $19.4\sqrt{2a} + 2 \leq 28\sqrt{a} + 2$ bits. A string is a code if it parses into two such codes of total weight at most a, followed by zeros. The multiplicity of the type i is the multiplicity listed for the weight $i + 1$, or 0. All of this is computed by scanning the string with counters of $O(\log(a+2))$ bits, rejecting as soon as a running total exceeds a. □

4.2 Cliques and their potential

The recursion acts on cliques, networks in which every pair of nodes shares a label of a prescribed weight.

Definition 4.2. Let $h \geq 0$ and $q = 2^h$. A *clique* $\mathcal{C}$ with *name length* h has nodes $\{0,1\}^h$, a *budget* $a_{uv} = a_{vu} \in \mathbb{N}$ for every unordered pair of distinct nodes, and for every node v a table $T_v \colon \prod_{u \neq v} \Lambda(a_{uv}) \to \mathbb{Z}$. Its value is

$$Z(\mathcal{C}) = \sum_\theta \prod_v T_v(\theta_v),$$

summed over one label $\theta_{uv} = \theta_{vu} \in \Lambda(a_{uv})$ for every pair, where $\theta_v = (\theta_{uv})_{u \neq v}$. Its *potential* is

$$\Phi(\mathcal{C}) = (q-1)^2 \sum_{\{u,v\}} a_{uv},$$

the sum running over unordered pairs.

Every label thus has weight equal to the budget of its pair. The pieces of Section 5 need this exactness, for their grade bound and for Lemma 5.2.

The potential controls the cost of writing the labels down. By Lemma 4.1 and Cauchy–Schwarz over the $\binom{q}{2}$ pairs, the labels of all pairs fit in

$$\sum_{\{u,v\}} w(a_{uv}) \leq 30\sqrt{\tbinom{q}{2} \sum a_{uv}} \leq 30\sqrt{\Phi(\mathcal{C})}$$

bits, so brute force over the labels of a clique uses an index of $O(\sqrt{\Phi})$ bits. Our potential is the square of the potential $q\|w\|_2$ of [Zhu26] (the number of nodes times the Euclidean norm of the vector of widths), written as a sum with a weight budget in place of each squared width. The factor $(q-1)^2$ rather than q^2 absorbs the factor by which packing scales weights (Lemma 6.1).

For the merge, it helps to write the two copies of a label separately, θ_{uv} in the argument of T_u and θ_{vu} in that of T_v. Let each copy range independently over the label set of its pair. Then $Z(\mathcal{C})$ is the sum over all copies of $\prod_v T_v$ times one equality test for each pair.

4.3 The first clique

The recursion starts from a clique built from the network, with potential $O(s^2)$. Fix a family of networks with logspace access at size s, and a network G in it with $M = O(s)$ edges and maximum degree $\Delta = O(\log s)$. We may assume that s is large. Let h_* be the largest power of 2 with $2^{3h_*} \le s$, and put $q_* = 2^{h_*}$. Then $q_*^3 \le s < q_*^6$, so $h_* = \Theta(\log s)$, and halving the name length repeatedly from h_* ends at 1.

Labels in the first clique encode strings of edge bits. A string $\theta \in \{0,1\}^b$ is encoded as the label $\mathrm{enc}(\theta) = (\sigma_1, \sigma_2)$ that has, for each position $\nu = 1, \dots, b$, one request of type $\nu - 1$, in σ_1 if $\theta_\nu = 1$ and in σ_2 if $\theta_\nu = 0$. Every string of b bits is encoded with weight $1+2+\cdots+b = b(b+1)/2$. A label of this weight is an encoding exactly when each type below b occurs once in its two multisets together, and then $\theta_\nu = \mu_{\sigma_1}(\nu - 1)$. In particular enc is injective.

The first clique groups the vertices of G into half of its q_* nodes, as in Williams's algorithm for 2-CSP [Wil05], and equality tests into the other half. A grouping that put edges inside a node would leave sums over those edges inside its table. To avoid these internal sums, we subdivide every edge by an equality test. A pair carrying b bits has budget $b(b+1)/2$, so concentrating edges on a few pairs increases the potential. For each pair of vertex groups, we therefore distribute its tests cyclically over the test nodes.

Lemma 4.3. There is a clique $\mathcal{C}_0$ with name length h_* and value $Z(G)$ such that:

(a) $\Phi(\mathcal{C}_0) = O\big((M + q_*\Delta + q_*^3)^2\big) = O(s^2)$;

(b) every table entry of $\mathcal{C}_0$ is a product, with no sum, of values of functions of G, tests of whether labels are encodings of strings, and tests of whether two bits agree;

(c) every budget of $\mathcal{C}_0$, and every factor of a table entry, given read access to the codes of its labels, can be computed in space $O(\log s)$ with logspace access to G, and every table entry has at most $s^{O(1)}$ factors.

The proof is in Appendix A. The constraint $q_*^3 \le s$ keeps the cost of the cyclic distribution, about q_* bits on each of the q_*^2 pairs, within $O(s)$ bits, and so within potential $O(s^2)$.

5 Crossing pieces

Merging rows or columns reduces the number of nodes, but a crossing still carries both diagonal labels across either cut. We seek a sum of pieces with a tradeoff between the message lengths needed for the two cuts. For diagonals of total weight H, we construct pieces Π_k, over even k with $|k| \le 2H$, that sum to the crossing and that factor across the row cut through messages of weight at most $H + k/2$ and across the column cut through messages of weight at most $H - k/2$ (Corollary 5.5). The two budgets add up to $2H$ whatever k, and Section 6 turns this into the halving of the potential.

Fix a rectangle with corners x, y in its upper row and u, v in its lower row, so that x, v are the ends of one diagonal and y, u of the other, as in Fig. 2. Each corner sees the label of its diagonal as a pair $(\mathrm{out}, \mathrm{in})$, and we identify a corner with this view, whose weight $\mathrm{wt}\,\mathrm{out} + \mathrm{wt}\,\mathrm{in}$ is that of

the label. We let the labels at x and v have weight a and those at y and u weight b, and put $H = a + b$. The crossing is $[x = v][y = u]$, where $[x = v]$ means that $\mathrm{out}_x = \mathrm{in}_v$ and $\mathrm{in}_x = \mathrm{out}_v$.

The row cut separates (x, y) from (u, v), and the column cut separates (x, u) from (y, v). A function of the four views *factors across a cut through messages* from a set $\mathcal{S}$ if it is a sum, over $m \in \mathcal{S}$, of a function of m and the views on one side times a function of m and the views on the other, as in a protocol in which the two sides agree on one message. When $\mathcal{S} = \{0,1\}^c$ this is Anthropic's notion of factoring through c bits [Ant26]. Each message contributes an outer product to the matrix indexed by the views on the two sides, so the matrix has rank at most $|\mathcal{S}|$. The recursion needs more than a rank bound. There are two cuts, with budgets traded between them, and a message becomes part of a new label that will be cut again. Messages must therefore be labels of controlled weight, and their factors must be computable in small space (Lemma 5.6).

The one-bit identity (1) shows why a sum helps. As 4×4 matrices with rows indexed by (x, y) and columns by (u, v), its first term $[x = u][y = v]$ is a permutation matrix of rank 4 and its second term an outer product of rank 1; across the column cut the two ranks are exchanged. The crossing itself has rank 4 across both cuts. For strings of λ bits, multiplying (1) over the coordinates gives 2^λ terms, each passing 2λ bits through the two cuts together instead of 4λ, but naming a term takes λ bits and wipes out the saving. Grouping the terms by the number of coordinates that take the first term gives polynomially many pieces [Ant26, Appendix H], and Anthropic's operators give better ones [Ant26, Appendix B].

We construct pieces from walks on the cube: the identity below splits an equality test at an intermediate type, whose weight the step lengths pay for. Summing routings made from these steps will give the pieces.

5.1 Walks on the cube

For $i, j \in \mathbb{N}$, let $i \oplus j$ be the bitwise exclusive or of their binary expansions, read as an integer, which is the dyadic sum of Walsh analysis [SWS90], and let $\langle i, j \rangle$ be the parity of the number of positions where both have a one. So $d(i, j) = i \oplus j$ is the Hamming distance in which bit e has length 2^e. Put

$$C_{i,j}(z) = (-1)^{\langle i,j \rangle}\, z^{d(i,j)},$$

a step of length $d(i, j)$ and sign $(-1)^{\langle i,j \rangle}$, and for a Laurent polynomial f write $\bar{f}(z) = f(1/z)$.

On types below 2^r, the matrix C is the tensor product over $e < r$ of $\left(\begin{smallmatrix} 1 & z^{2^e} \\ z^{2^e} & -1 \end{smallmatrix}\right)$: a Walsh–Hadamard matrix $\mathcal{H}$ with entry (i, j) multiplied by $z^{d(i,j)}$ [SWS90]. Each factor squares to $(1 + z^{2^{e+1}})I$, so the product squares to

$$\prod_{e<r} (1 + z^{2^{e+1}})\, I = (1 + z^2 + \cdots + z^{2^{r+1}-2})\, I.$$

At $z = 1$ this is $\mathcal{H}^2 = 2^r I$. As r grows, the scalar converges coefficientwise to $1/(1 - z^2)$, giving Lemma 5.1, which we prove directly below.

The least significant bit of $i \oplus j$ is the parity of $i + j$, so $d(i, j) \equiv i + j \pmod 2$. Beyond this parity, the accounting below uses only

$$d(i, j) \le i + j \qquad \text{and} \qquad j \le i + d(i, j). \tag{2}$$

The second inequality says that the weight $j + 1$ of a type is at most the weight $i + 1$ of another type plus their distance.

Lemma 5.1. For all $i, j \in \mathbb{N}$, $(1-z^2)\sum_{n\in\mathbb{N}} C_{i,n}(z)\,C_{n,j}(z) = [i=j]$ in the ring $\mathbb{Z}((z))$ of Laurent series.

Proof. The term for n has exponent $d(i,n)+d(n,j) \geq n-i$ by (2), so each coefficient of the series receives finitely many terms; the sign of the term is $(-1)^{\langle i\oplus j, n\rangle}$. Suppose $i \neq j$. Fix a position where i and j differ, and let n' be n with that bit flipped. In that position exactly one of $i\oplus n$ and $n\oplus j$ has a one, both for n and for n', so the flip moves the length of that bit from one step to the other and keeps the exponent. The sign changes, because $i\oplus j$ has a one in that position. The terms of each exponent therefore cancel in pairs. If $i=j$, the term for n is $z^{2(i\oplus n)}$, and $i\oplus n$ runs over $\mathbb{N}$ as n does, so the sum is $\sum_{c\geq 0} z^{2c} = 1/(1-z^2)$. □

5.2 Routing pieces

The cut carries one message for the rectangle, with no mark recording which corner sent a request or which corner will receive it. Consider a single request between the two rows. To cross the cut it meets an intermediate type n, with a half-rule at each end: a step $C_{i,n}$ or a copy $[i=n]$. The products of the two half-rules are summed over n (Fig. 2(b)). Sharing the message joins every upper corner to every lower corner, with a rule given by the product of their half-rules. If all corners copied, the sides would carry equality tests as well as the diagonals. Instead, let the corners y and u copy, and let x and v step, putting the factor $1-z^2$ of Lemma 5.1 at x. With $I = ([i=j])$, the upper corners x, y indexing rows and the lower corners u, v indexing columns, the four ways to cross multiply out as

$$\begin{pmatrix} (1-z^2)\,C \\ I \end{pmatrix} \begin{pmatrix} I & C \end{pmatrix} = \begin{pmatrix} (1-z^2)\,C & I \\ I & C \end{pmatrix}, \tag{3}$$

where the upper-right entry follows from Lemma 5.1. These choices give the diagonals their equality tests and determine the vertical rules: xu gets $(1-z^2)\,C$ and yv gets C. The column cut, with x, u indexing rows, y, v indexing columns and $1/z$ in place of z so that horizontal lengths count negatively, gives xy the rule $(1-z^{-2})\,\bar{C}$ and uv the rule $\bar{C}$. Since $C_{i,j} = C_{j,i}$, no rule depends on the direction of a request. The factors $1-z^{\pm 2}$ make the two side rules at x vanish at $z=1$, leaving only the diagonal.

Now let all the requests at the four corners travel at once. An occurrence of a request at a corner is a source if the corner sends it and a target if it receives it. At each corner, we list sources by type, numbering the occurrences of each type. Sources of the same type are distinct; targets of the same type are interchangeable and specified only by their multiplicities. A *routing* ρ sends every source at every corner to a target at another corner, meeting every target multiplicity exactly: for every corner c and type j, exactly $\mu_{\mathrm{in}_c}(j)$ sources are sent to the type j at c. A source together with its target is a request of ρ, and it travels along the pair of corners at its two ends; along a diagonal it must join equal types.

With i the type of the end at x or v and j that at y or u, the rules of the sides, from (3), are

$$R_{xu} = (1-z^2)\,C_{i,j}, \qquad R_{vy} = C_{i,j}, \qquad R_{xy} = (1-z^{-2})\,\bar{C}_{i,j}, \qquad R_{vu} = \bar{C}_{i,j}$$

(Fig. 2), and a request along a diagonal has the rule 1. The value $\mathrm{val}(\rho)$ of the routing is the product of the rules of its requests, so in its exponents, length along the vertical sides xu, yv counts positively and length along the horizontal sides xy, uv negatively. Let

$$\Pi_{x,y,u,v}(z) = \sum_{\rho} \mathrm{val}(\rho),$$

summed over all routings. There are finitely many routings, so $\Pi_{x,y,u,v}$ is a Laurent polynomial. Its coefficients $\Pi_k = [z^k]\Pi_{x,y,u,v}$ are the pieces, and k is the grade of Π_k.

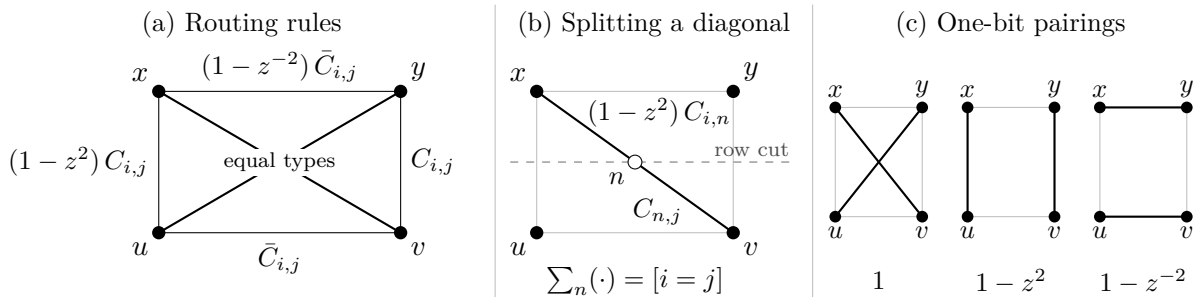


Figure 2: (a) Routing rules; transposition exchanges vertical and horizontal sides and replaces z by $1/z$. (b) Splitting the diagonal xv at an intermediate type n; summing over n recovers equality (Lemma 5.1). (c) The three possible pairings for one-bit labels, with their values when the endpoint directions permit them. At $z = 1$ only the diagonal pairing remains.

Every grade is even and has absolute value at most $2H$. Indeed, every exponent of the rule of a request between types i and j is congruent to $(i + 1) + (j + 1)$ modulo 2 and at most $(i + 1) + (j + 1)$ in absolute value. On a side the exponents are $\pm d(i, j)$, and also $\pm(d(i, j) + 2)$ at x, with $d(i, j) \equiv i + j$ and $d(i, j) + 2 \le (i + 1) + (j + 1)$ by (2); on a diagonal the exponent is 0 and $i = j$. Every occurrence of a request at a corner is an end of exactly one request of ρ, and the occurrences at the four corners weigh $2a + 2b = 2H$ in all. Hence $\Pi_k = 0$ unless k is even and $|k| \le 2H$.

Lemma 5.2. If $\operatorname{wt} x = \operatorname{wt} v$, then $\Pi_{x,y,u,v}(1) = [x = v][y = u]$.

Proof. At $z = 1$ the rules of the two sides at x vanish, because of their factors $1 - z^{\pm 2}$, so $\operatorname{val}(\rho)(1) = 0$ for every routing with a request along a side at x. So $\Pi_{x,y,u,v}(1)$ is the sum of $\operatorname{val}(\rho)(1)$ over the routings in which every request at x travels along the diagonal xv.

In such a routing every source at x goes to a target of the same type at v, and every target at x receives a source of the same type from v. These requests use occurrences at v of total weight $\operatorname{wt} x = \operatorname{wt} v$, and every request has positive weight, so they use all the occurrences at v. Hence $\operatorname{out}_x = \operatorname{in}_v$ and $\operatorname{in}_x = \operatorname{out}_v$, and no request travels along vy or vu. The requests at y and u must then travel along yu, which forces $\operatorname{out}_y = \operatorname{in}_u$ and $\operatorname{in}_y = \operatorname{out}_u$. Conversely, if $x = v$ and $y = u$, there is exactly one such routing, which sends every source to its own type at the opposite corner, and its value is 1. □

Lemma 5.3. $\Pi_{x,y,u,v}(z) = \Pi_{x,u,y,v}(1/z)$.

Proof. On the right, u plays the part of the upper corner y and y that of the lower corner u: we transpose the rectangle, reflecting it in its diagonal xv. A routing for (x, y, u, v) is also a routing for (x, u, y, v), since no source or target changes. Its requests keep their ends, but vertical sides become horizontal and horizontal sides vertical, so the rule of every request becomes the same rule in $1/z$, and $\operatorname{val}(\rho)(z)$ becomes $\operatorname{val}(\rho)(1/z)$. □

For one-bit labels, the pieces can be written explicitly. In this paragraph let x, y, u, v denote the encoded bits: a 1 is a source at an upper corner and a target at a lower corner. Every request

has type 0. A pairing is allowed when its endpoint bits agree on a diagonal or vertical side, and differ on a horizontal side. The three possible pairings (Fig. 2(c)) therefore give

$$\Pi(z) = [x = v][y = u] + (1 - z^2)[x = u][y = v] + (1 - z^{-2})[x \neq y][u \neq v].$$

Thus $\Pi_2 = -[x = u][y = v]$, $\Pi_{-2} = -[x \neq y][u \neq v]$, and $\Pi_0 = 2[x + y = u + v]$: the constant coefficient counts the two matchings whenever the corners hold two sources and two targets. The row and column ranks of these pieces over $\mathbb{Q}$ are $(4, 1)$, $(1, 4)$ and $(3, 3)$, respectively, compared with $(4, 4)$ for the crossing. For Π_0, the three rank-one blocks are indexed by $x + y = u + v$ across rows and by $x - u = v - y$ across columns. These pieces differ from the two terms of (1).

When all four encoded bits are 1, $\Pi(z) = 2 - z^2$ and $\Pi_2 = -1$. After transposition u is an upper target and y a lower source, so the encoded bits become $(1, 0, 0, 1)$, giving $2 - z^{-2}$.

5.3 Factoring across the two cuts

To factor a piece across the row cut, we cut every request that crosses it. A message $m = (\eta, \zeta)$ is a label whose first multiset η is sent downward across the row cut and whose second multiset ζ is sent upward. A request that crosses the row cut meets the message at an intermediate type n, and at its corner, of type i, it has the half-rule of (3): the step $(1 - z^2)\, C_{i,n}$ at x or $C_{i,n}$ at v, and the copy $[i = n]$ at y or u.

Cutting every crossing request leaves a smaller routing problem on each side. An upper half-routing for views x, y and a message m sends every source at x or y either to a target at the other upper corner or downward as a request of η, and sends every request of ζ to a target at x or y. It must give x and y exactly their incoming multisets, and the requests sent downward must have exactly the multiplicities of η. So the requests of η are interchangeable targets here, while those of ζ are numbered sources, as at a corner.

The value of an upper half-routing is the product of R_{xy} over the requests between x and y and of the half-rules over the requests of the message, and $N^{\uparrow}(x, y; m)$ is the sum of these values. A lower half-routing for u, v and m is defined in the same way, with the roles of η and ζ exchanged and the rule R_{vu} between u and v; $N^{\downarrow}(u, v; m)$ is the sum of their values.

Lemma 5.4. If x, v have weight a and y, u have weight b, then

$$\Pi_{x,y,u,v}(z) = \sum_m N^{\uparrow}(x, y; m)\, N^{\downarrow}(u, v; m)$$

in $\mathbb{Z}((z))$, and every exponent of $N^{\uparrow}(x, y; m)$ and of $N^{\downarrow}(u, v; m)$ is at least $\operatorname{wt} m - H$.

Proof. We first bound the exponents. Fix an upper half-routing for m, and give every occurrence of type i at x or y its weight $i + 1$. These occurrences weigh H in all, and each lies in exactly one factor of the value. A request between x and y, of types i and j, has exponents at least $-d(i, j) - 2 \geq -(i + 1) - (j + 1)$, by the first inequality of (2). A request of the message of type n, met at a corner occurrence of type i, has a half-rule whose exponents are at least $(n + 1) - (i + 1)$: for a step this is the second inequality of (2), and a copy has $n = i$. Every request of the message is met exactly once, so every exponent of $N^{\uparrow}(x, y; m)$ is at least $\operatorname{wt} m - H$, and the same argument applies to $N^{\downarrow}(u, v; m)$. In particular only finitely many messages contribute to each power of z in $\sum_m N^{\uparrow}(x, y; m)\, N^{\downarrow}(u, v; m)$.

Now glue. Given an upper and a lower half-routing for m, join the requests of type n sent downward in the upper one, taken in the order of their sources, one by one to the requests of

type n of η in the lower one, and likewise upward. This gives an assignment of every source at a corner to a target at another corner, meeting every target multiplicity, together with an intermediate type for every request between the rows. Cutting in the same order inverts the gluing, so every assignment with intermediate types arises from exactly one pair of half-routings, and the value of the pair is the product of the rules of the requests inside the rows and of the half-rules of the requests between them.

Such an assignment need not be a routing: a request along a diagonal may join two different types. Summing over the intermediate types repairs this. For each request between the rows, the sum of the products of its two half-rules is the entry of (3) for its two corners: the rule of its side on xu or yv, and the test $[i = j]$ on a diagonal, by Lemma 5.1 on xv. So the sum over intermediate types turns each assignment into the product of the rules of its requests, which vanishes unless every request along a diagonal joins equal types. The assignments that survive are exactly the routings, and the total is $\Pi_{x,y,u,v}(z)$. The sums may be rearranged in $\mathbb{Z}((z))$, since the exponents of $(1 - z^2)C_{i,n}$ are at least $n - i$ and each power of z receives finitely many messages. □

Corollary 5.5. If x, v have weight a and y, u have weight b, then for every even k with $|k| \le 2H$,

$$\Pi_k = \sum_{m \in \Lambda_{\le}(H+k/2)} \sum_c [z^{k-c}] N^{\uparrow}(x, y; m) \cdot [z^c] N^{\downarrow}(u, v; m) \qquad \text{(row cut)},$$

$$\Pi_k = \sum_{m \in \Lambda_{\le}(H-k/2)} \sum_c [z^{-k-c}] N^{\uparrow}(x, u; m) \cdot [z^c] N^{\downarrow}(y, v; m) \qquad \text{(column cut)},$$

where only c with $\operatorname{wt} m - H \le c \le H \pm k - \operatorname{wt} m$ contribute, with $+$ for the row cut and $-$ for the column cut. In particular $|c| \le 3H$.

Proof. Put $\delta = \operatorname{wt} m - H$. By Lemma 5.4, a pair of monomials of $N^{\uparrow}(x, y; m)$ and $N^{\downarrow}(u, v; m)$ with exponents $r - c$ and c has $c \ge \delta$ and $r - c \ge \delta$, that is, $\delta \le c \le r - \delta$. This range is empty unless $\operatorname{wt} m \le H + r/2$, and since $\delta \ge -H$ and $|r| \le 2H$ it lies in $[-H, 3H]$. Taking $r = k$ gives the row cut. By Lemma 5.3, Π_k is the coefficient of z^{-k} in $\Pi_{x,u,y,v}$, whose views have the same weights, and taking $r = -k$ there gives the column cut. □

The example after Lemma 5.3 shows the two cuts producing the same piece, $\Pi_2 = -1$, with $H = 2$. On the row cut, x and y receive nothing, so both of their requests cross downward: y copies type 0, and x steps to some type n, with the half-rule $(1 - z^2)\, z^n$. The message $\eta = \{0, n\}$ weighs $n + 2 \le H + 1$, so $n \le 1$. For $n = 0$ the lower half sends the two requests of η to u and v in either order, so $N^{\uparrow} = 1 - z^2$ and $N^{\downarrow} = 2$. For $n = 1$ the request of type 1 cannot be copied at u and must go to v, so $N^{\uparrow} = (1 - z^2)\, z$ and $N^{\downarrow} = z$; type 1 occurs at no corner, yet it contributes. On the column cut the cap is $H - 1 = 1$. Each column holds one source and one target, so a nonempty message would cross in both directions and weigh at least 2. Only the empty message remains, with $N^{\uparrow}(x, u; \emptyset) = 1 - z^{-2}$ and $N^{\downarrow}(y, v; \emptyset) = 1$. The contributions at splits $c = 0, 1$ are $-2, 1$ on the row cut and $-1, 0$ on the column cut. Both sum to -1, but choosing the column cut at $c = 0$ and the row cut at $c = 1$ would give 0. So a merge may choose its cut by the grades, but must keep it for all the splits.

The full index of each factorization in Corollary 5.5 is the pair (m, c), where the split c has $O(\log(H + 2))$ bits; the merge of Section 6.1 records c in its choice and packs m into the label shared by two rows.

5.4 Computing the factors

A table entry of a merged clique needs the factors $N^{\uparrow}$ and $N^{\downarrow}$ only at single exponents. They are signed counts as well, and a short recursion computes them.

Lemma 5.6. Let $W = \operatorname{wt} x + \operatorname{wt} y + \operatorname{wt} m$. For every integer c, the coefficient $[z^c]N^{\uparrow}(x, y; m)$ is the value of a formula with ℓ-cost $O\big(\sqrt{W} + (\ell + \log(W+2))\log(W+2)\big)$ for every $\ell \geq 1$, which is $O(\sqrt{W} + \log^2(W+2))$-explicit given read access to the codes of x, y and m. The same holds for $[z^c]N^{\downarrow}(u, v; m)$, with $W = \operatorname{wt} u + \operatorname{wt} v + \operatorname{wt} m$.

The idea is a balanced split. The factor $N^{\uparrow}(x, y; m)$ is a signed count of maps from the sources out_x, out_y and ζ to the targets in_x, in_y and η. The formula splits such a map at the first source at which the running weight of the sources and their targets reaches half of the total; with that source and its target set aside, the parts before and after it weigh at most $W/2$. The targets used by the first half form a multiset of weight at most W, which takes $O(\sqrt{W})$ bits by Lemma 4.1, and the weight halves at every level. The proof is in Appendix A.

6 Halving node names

Each piece of a crossing factors across either cut. For each list of grades, we must choose one cut for the whole clique and show that the resulting messages fit into a smaller clique, so that the construction can be repeated. Each step takes a clique $\mathcal{C}$ with even name length h and rewrites its value as

$$Z(\mathcal{C}) = \sum_{\iota} Z(\mathcal{C}_\iota), \qquad \frac{\Phi(\mathcal{C}_\iota)}{h/2} \leq \frac{\Phi(\mathcal{C})}{h},$$

where ι chooses a piece and a split in every rectangle, and each $\mathcal{C}_\iota$ has name length $h/2$. Repeating the step $\log h_*$ times leaves two nodes. We then unfold the clique values and table entries into a formula and use the potential bound to control the indices on every path.

6.1 One merge

Let $\mathcal{C}$ be a clique with even name length $h \geq 2$, and put $g = 2^{h/2}$, so that $q = g^2$. Write each node as (A, B), where $A, B \in \{0,1\}^{h/2}$ are the first and second halves of its name, and call A its row and B its column. In the lexicographic order of names, $(A, B) < (A', B')$ whenever $A < A'$, and $(A, B) < (A, B')$ whenever $B < B'$.

A pair of nodes in the same row is horizontal, one in the same column is vertical, and pairs of either kind are straight. For rows $A < A'$ and columns $B < B'$, the rectangle $R = \{A, A'\} \times \{B, B'\}$ has the corners

$$x = (A, B), \qquad y = (A, B'), \qquad u = (A', B), \qquad v = (A', B'),$$

and the diagonals $\{x, v\}$ and $\{y, u\}$. Every pair of nodes in different rows and different columns is a diagonal of exactly one rectangle. The copies of the diagonal labels at the four corners, seen from their corners, are the views of Section 5, and H_R is the sum of the budgets of the two diagonals.

A choice ι picks for every rectangle R two integers, an even grade k_R with $|k_R| \leq 2H_R$ and a split c_R with $|c_R| \leq 3H_R$. To see what a merged clique must hold, expand $Z(\mathcal{C})$. Writing out the copies, $Z(\mathcal{C})$ is the sum over all copies of $\prod_v T_v$ times the equality tests of all pairs. The tests of

the two diagonals of each rectangle multiply to its crossing. Every copy has weight equal to the budget of its pair, so Lemma 5.2 and the parity of grades express the crossing as $\sum_k \Pi_k$, over even k with $|k| \le 2H_R$. Expanding the product over rectangles gives $Z(\mathcal{C}) = \sum_{\mathbf{k}} Z_{\mathbf{k}}$, over the lists $\mathbf{k} = (k_R)_R$ of grades, where $Z_{\mathbf{k}}$ is the sum over all copies of $\prod_v T_v$ times the tests of the straight pairs times $\prod_R \Pi_{k_R}$.

Fix $\mathbf{k}$ and take the row cut of every piece. The horizontal tests stay inside their rows. A vertical test with budget a compares copies p, p' held in two different rows, and we write it as

$$[p = p'] = \sum_{\lambda \in \Lambda(a)} [p = \lambda][p' = \lambda].$$

By Corollary 5.5, each Π_{k_R} is a sum over messages $m_R \in \Lambda_{\le}(H_R + k_R/2)$ and splits c_R with $|c_R| \le 3H_R$, where the splits outside the range of Corollary 5.5 give 0. So two rows $A < A'$ share a tuple $\chi_{A,A'}$ of g labels λ, one for the vertical pair in each column, and $\binom{g}{2}$ messages m_R, one for each rectangle on the two rows. The tuple ranges over the product $\mathcal{S}_{A,A'}$ of the sets $\Lambda(a)$ of the vertical pairs and $\Lambda_{\le}(H_R + k_R/2)$ of the rectangles, where here and below R is the rectangle with rows A, A' and columns B, B'. Once the splits $\mathbf{c} = (c_R)_R$ and the tuples are fixed, every remaining factor involves the copies of a single row. Write χ_A for the tuples shared by A with the other rows, and put

$$\Xi^{\mathbf{c}}_A(\chi_A) = \sum_{\xi} \prod_B T_{(A,B)}(\xi_{(A,B)}) \cdot \prod_{\text{horizontal}} [\text{copies agree}] \cdot \prod_{\text{vertical}} [\text{copy} = \lambda] \cdot \prod_R \Psi_R, \tag{4}$$

the sum over the settings $\xi = (\xi_v)_{v \in A \times \{0,1\}^{h/2}}$ of the old nodes of row A, that is, over all copies held in row A, each in the label set of its pair. The horizontal tests compare the two copies of each pair inside the row. The vertical tests compare the copy at (A, B) of the label of a vertical pair with the label λ of that pair in χ_A. The last product has one factor for each rectangle R with A among its rows, with the message m_R from χ_A and the views read from ξ:

$$\Psi_R = [z^{k_R - c_R}] N^{\uparrow}(x, y; m_R) \qquad \text{if } A \text{ is its upper row,}$$
$$\Psi_R = [z^{c_R}] N^{\downarrow}(u, v; m_R) \qquad \text{if } A \text{ is its lower row.}$$

Summing the copies row by row gives

$$Z_{\mathbf{k}} = \sum_{\mathbf{c}} \sum_{(\chi_{A,A'})_{A<A'}} \prod_A \Xi^{\mathbf{c}}_A(\chi_A), \tag{5}$$

over $\chi_{A,A'} \in \mathcal{S}_{A,A'}$ for every pair of rows. For each $\mathbf{c}$ the inner sum is already the value of a clique on the rows, except that two rows share a tuple instead of a label. The column cut gives the same with rows and columns exchanged: columns $B < B'$ share the labels of their horizontal pairs and the messages $m_R \in \Lambda_{\le}(H_R - k_R/2)$ of their rectangles, with the first multiset of m_R sent from B to B'; the factors are $\Psi_R = [z^{-k_R - c_R}] N^{\uparrow}(x, u; m_R)$ in the left column and $\Psi_R = [z^{c_R}] N^{\downarrow}(y, v; m_R)$ in the right.

To apply the construction again, we encode each tuple as one label of exact weight and account for the increase in weight. Put $L = 1 + g + \binom{g}{2}$, one residue modulo L for each entry of a tuple and the residue 0 for padding. Between rows $A < A'$, number the vertical pairs by their columns and then the rectangles by their pairs of columns in lexicographic order, as streams $\nu = 1, \dots, L-1$, and let a request of type i in stream ν become a packed request of type $Li + \nu$, in the same direction. Its weight satisfies

$$Li + \nu + 1 \le L(i + 1). \tag{6}$$

The downward multiset of the new label packs the first multisets of the labels λ and the multisets η_R, and the upward multiset packs their second multisets and the multisets ζ_R; requests of type 0 in the downward multiset pad the label up to its budget. A label is valid if its only requests of type divisible by L are padding. A valid label is unpacked by removing the padding and dividing every other type by L, with quotient i and remainder ν. So the valid labels of a given weight correspond one-to-one to the tuples whose packed requests weigh at most that weight. By (6), the row budgets and column budgets

$$a^{\mathrm{row}}_{A,A'} = L\Big(\sum_{B} a_{(A,B),(A',B)} + \sum_{B<B'} (H_R + k_R/2)\Big),$$
$$a^{\mathrm{col}}_{B,B'} = L\Big(\sum_{A} a_{(A,B),(A,B')} + \sum_{A<A'} (H_R - k_R/2)\Big)$$

leave room for every tuple of $\mathcal{S}_{A,A'}$ and of its column counterpart. They depend only on the grades.

We orient the whole clique by rows or by columns, comparing the two potentials.

Lemma 6.1. For every choice ι, let Φ_{row} and Φ_{col} be the potentials of cliques with name length $h/2$ and the budgets a^{row} and a^{col}. Then $\Phi_{\mathrm{row}} + \Phi_{\mathrm{col}} \le \Phi(\mathcal{C})$.

Proof. A clique with name length $h/2$ has g nodes. Every vertical pair and every rectangle has exactly one pair of rows, and every horizontal pair and every rectangle exactly one pair of columns, so

$$\Phi_{\mathrm{row}} + \Phi_{\mathrm{col}} = (g-1)^2 L\Big(\sum_{\text{straight}} a + \sum_{R} 2H_R\Big) \le (g-1)^2 \cdot 2L \sum_{\{u,v\}} a_{uv},$$

because every pair is straight or a diagonal of exactly one rectangle. Finally $2L = g^2 + g + 2 \le (g+1)^2$, and $(g-1)^2(g+1)^2 = (q-1)^2$. □

Without pieces, two rows would share g^2 labels, and packing would use $L_0 = g^2 + 1$. The same calculation gives only $\Phi_{\mathrm{row}} + \Phi_{\mathrm{col}} \le 2\Phi$, since $L_0 \le (g+1)^2$. The pieces replace each rectangle's two labels by one message, reducing the packing factor to $L = 1 + g + \binom{g}{2}$, asymptotically half as large, while their budgets across the two orientations still add to $2H_R$. This gives the uniform halving of the potential.

With the budgets fixed, we must define the merged tables so that packing preserves the clique value. Fix a choice ι. The row clique $\mathcal{C}^{\mathrm{row}}_\iota$ has name length $h/2$, the rows as nodes, and budgets a^{row}. Its table at row A is 0 if one of its labels is invalid, and otherwise the sum (4) at the tuples χ_A unpacked from its labels, with every factor Ψ_R multiplied by the weight test $[\mathrm{wt}\, m_R \le H_R + k_R/2]$. On tuples in $\mathcal{S}_{A,A'}$ this is $\Xi^{\mathbf{c}}_A(\chi_A)$, and outside them it vanishes: a vertical label of the wrong weight fails its test against every copy, which has the exact weight of its pair, and a heavier message fails its weight test.

The weight tests do not change the clique value: for a heavier message, the product of its two coefficients already vanishes by Lemma 5.4, although either coefficient alone may be nonzero. Including the tests makes each table vanish outside the prescribed tuple sets and bounds the message weights passed to the coefficient formulas (Lemma 6.3). The column clique $\mathcal{C}^{\mathrm{col}}_\iota$ is built in the same way from the column cut, with nodes the columns and budgets a^{col}.

Lemma 6.2. Let $\mathcal{C}$ be a clique with even name length $h \geq 2$, and let o assign an orientation in $\{\text{row}, \text{col}\}$ to every list of grades. Then

$$Z(\mathcal{C}) = \sum_{\iota} Z(\mathcal{C}_\iota^{o(\iota)}),$$

where $o(\iota)$ is the orientation of the grades of ι. An entry of a table of $\mathcal{C}_\iota^{o(\iota)}$ is a sum, over the copies held in one row or column, of a product of entries of tables of $\mathcal{C}$, equality tests, weight tests, and coefficients of the factors $N^{\uparrow}$ and $N^{\downarrow}$.

Proof. Suppose $o(\mathbf{k}) = \text{row}$, and fix the splits $\mathbf{c}$ of a choice ι with grades $\mathbf{k}$. A term of $Z(\mathcal{C}_\iota^{\text{row}})$ assigns a label to every pair of rows, and its tables vanish unless every label is valid and unpacks into $\mathcal{S}_{A,A'}$. On these labels, unpacking is a bijection onto $\prod_{A<A'} \mathcal{S}_{A,A'}$, and the tables are the factors $\Xi_A^{\mathbf{c}}$. So $Z(\mathcal{C}_\iota^{\text{row}})$ is the inner sum of (5), and summing over the splits gives $Z_{\mathbf{k}}$. The case $o(\mathbf{k}) = \text{col}$ is the same with the column cut. Summing over $\mathbf{k}$ gives $Z(\mathcal{C})$. □

The orientation is chosen once for each list of grades, after all the pieces are fixed. It cannot be chosen rectangle by rectangle, since a term must become a clique on rows or a clique on columns as a whole, and it cannot depend on the splits, since the two cuts reach the same piece only after the sum over the splits (the example after Corollary 5.5). The budgets of $\mathcal{C}_\iota^{o(\iota)}$ depend only on the grades of ι and the budgets of $\mathcal{C}$, never on the splits or the tables. With the orientation of smaller potential for each list of grades, Lemma 6.1 gives

$$\Phi(\mathcal{C}_\iota^{o(\iota)}) \leq \tfrac{1}{2}\Phi(\mathcal{C}), \qquad \text{hence} \qquad \frac{\Phi(\mathcal{C}_\iota^{o(\iota)})}{h/2} \leq \frac{\Phi(\mathcal{C})}{h}.$$

6.2 Unfolding the recursion

By Lemma 6.2, a clique value is a sum of merged clique values, and a merged table entry is a sum, over the copies in one row or column, of products of old entries, tests and coefficients. Whenever a sum ranges over labels, its index contains their codes from Lemma 4.1. The weight budgets bound the lengths of these codes, and the merges control the budgets (Lemma 6.3).

We start from the clique $\mathcal{C}_0$ of Lemma 4.3 and apply Lemma 6.2 with name lengths $h_*, h_*/2, \ldots, 2$, choosing for each list of grades the orientation of smaller potential, and the rows in case of a tie. Put $\Phi_* = \Phi(\mathcal{C}_0)$. Every budget of every clique below is at most its potential, and so at most Φ_*. Every grade and split is an integer of absolute value at most $6\Phi_*$, which we write in a field of fixed length $O(\log s)$; since $\Phi_* \leq s^3$ for large s by Lemma 4.3(a), fields of $3\lceil \log s \rceil + 5$ bits suffice.

The recursion unfolds into a formula in the sense of Definition 3.1, with the following nodes.

- The value of a clique with name length $h \geq 2$ is a sum node over the choices ι, written as two fields for each rectangle in a fixed order. The child for ι is the formula for the value of $\mathcal{C}_\iota^{o(\iota)}$, and strings that name no choice lead to the leaf 0.
- The value of a clique with name length 1, which has two nodes and one pair, is a sum node over the code of the label of that pair (Lemma 4.1), above a product node over its two nodes, whose children are the formulas for the two table entries. Strings that are not codes of labels of the right weight lead to the leaf 0.

- An entry of a table of a clique produced by a merge is the leaf 0 if one of its labels is not valid. Otherwise it is a sum node over the codes of the copies in ξ above a product node over the factors of (4), with 1 at unused indices; strings that are not codes of labels of the right weights lead to the leaf 0. The copies in ξ are listed by their nodes and then by their partners, and the factors in a fixed order. A factor is the formula for an entry of a table of the parent clique, at the argument ξ_v; a leaf, for an equality test; or, for a factor Ψ_R, the leaf 0 if the message fails the weight test and otherwise the formula of Lemma 5.6 for the coefficient.
- An entry of a table of $\mathcal{C}_0$ is a product node whose children are leaves, as in Lemma 4.3(b).

By Lemmas 4.3 and 6.2, the root has the value $Z(G)$. A table entry refers only to entries of the parent clique, never to the value of a clique, so every root-to-leaf path is V-shaped (Fig. 1). It descends through the choices at name lengths $h_*, h_*/2, \dots, 2$, sums over the last label at name length 1, and climbs through one chain of table entries. It may end in the recursion of Lemma 5.6.

6.3 Index lengths along a path

The potential limits the size of each label, but the evaluator holds several indices at once. We must bound their total cost along a path. We estimate the clique-value and table-entry parts separately, then account for the possible coefficient recursion at the end.

Lemma 6.3. Put $F = s/\sqrt{h_*}$. Every clique with name length h in the formula has potential at most $\Phi_* h/h_*$, and for every $\ell = O(\log s)$ the formula has ℓ-cost $O(F)$.

Proof. The bound on the potential follows from $\Phi(\mathcal{C}')/(h/2) \le \Phi(\mathcal{C})/h$ at every merge, by induction from $\mathcal{C}_0$. With $\Phi_* = O(s^2)$ it gives $\sqrt{\Phi} = O(F\sqrt{h})$ at name length h, and every budget is at most $\Phi/(q-1)^2$.

Consider the downward phase of a path first. It has one sum node over the choices for each name length $h = h_*, \dots, 2$. There are fewer than $q^2 \le q_*^2$ rectangles, each with two fields of $O(\log s)$ bits, so these indices have $O(q_*^2 \log s \log h_*)$ bits together, which is $o(F)$ because $q_*^2 \le s^{2/3}$. The sum node at name length 1 ranges over the codes of one label of weight $\Phi \le \Phi_*/h_*$, of $O(\sqrt{\Phi_*/h_*}) = O(F)$ bits.

In the upward phase, a table entry of a clique produced from a parent with name length $h \ge 2$ sums over the copies held in one row or column of the parent. That row or column has $g = 2^{h/2}$ old nodes and fewer than g^3 copies. Each pair holds at most two of these copies, so their budgets add up to at most $2\Phi/(q-1)^2 \le 8\Phi/q^2$, and by Lemma 4.1 and Cauchy–Schwarz the index has at most

$$30\sqrt{g^3 \cdot 8\Phi/q^2} = O(\sqrt{\Phi}\, 2^{-h/4}) = O\Big(\frac{F}{h}\Big)$$

bits, because $h^{3/2}2^{-h/4}$ is bounded. At most one such entry is open for each power of two h with $2 \le h \le h_*$, and $\sum_h 1/h < 1$ over these h, so together they have $O(F)$ bits.

Finally, the path may end in the recursion for a coefficient of $N^{\uparrow}$ or $N^{\downarrow}$ inside a table entry from a parent with name length h. The weight test has then passed, so the weight in Lemma 5.6 is $W \le H + \operatorname{wt} m \le 3H \le 24\Phi/q^2$, and the recursion adds $O(\sqrt{W} + \log^2 s) = O(F)$ bits. It remains to charge ℓ for every inner node and ℓw for every product node of index length w. A path has $O(\log h_*)$ inner nodes outside the recursion and $O(\log s)$ inside it. The product nodes of the table entries have indices of $O(h)$ bits, $O(\log s)$ over all name lengths, and those of the recursion have index length 2. An entry of $\mathcal{C}_0$ is a product over the vertices and edges of a group,

with an index of $O(\log s)$ bits because G has $s^{O(1)}$ vertices (Definition 2.1). So these charges add $O(\ell \log s) = O(\log^2 s) = o(F)$. □

6.4 Proof of Theorem 2.3

The path bound controls the evaluator's stack. To apply Proposition 3.2, we must also recover each addressed node in $O(F)$ space. Given an address, a machine follows it from the root and computes the budgets of the cliques along the path, the offsets of codes inside ξ and the orientations, which depend only on these budgets and the chosen grades. The leaves are evaluated with Lemmas 4.1 and 4.3 and the packing map (6), and the coefficients with Lemma 5.6. The proof of the following lemma, in Appendix A, keeps the budget arrays of the $O(\log h_*)$ cliques on the path and reads labels and messages inside packed labels on demand.

Lemma 6.4. The formula is S-explicit for some $S = O(F)$.

Proof of Theorem 2.3. By Definition 2.1, $0 \le Z(G) < 2^{s^{A_1}}$ for a constant A_1, so bit j of $Z(G)$ is 0 for $j \ge s^{A_1}$, which the machine detects in space $O(\log s)$. For $j < s^{A_1}$, take $\ell = \lceil A_2 \log s \rceil$ for a constant A_2 such that j and every index length of the formula are below 2^ℓ. By Lemmas 6.3 and 6.4, the formula has ℓ-cost $O(F)$ and is S-explicit with $S = O(F)$; its leaves have absolute value at most $s^{O(1)}$, and the input has length $s^{O(1)}$. So Proposition 3.2 computes bit j in space $O(F)$ and time $2^{O(F)}$, and $F = O(s/\sqrt{\log s})$. □

7 Counting classes and satisfiability

Computing the bits of an exact count allows threshold, parity, modular and equality tests. The distinction between choice strings and computations now matters: probabilistic acceptance uses choice strings of a fixed length, while the other counting classes below count computations.

A probabilistic machine is a nondeterministic machine that flips a fair coin at each step with two applicable transitions. If all its computations halt within $\bar{t}$ steps, it accepts x with probability $2^{-\bar{t}}$ times the number of accepting choice strings in $\{0,1\}^{\bar{t}}$. A language $\mathcal{L}$ is in $\mathsf{PPTIME}[t]$ if a probabilistic machine with running time $O(t)$ accepts exactly the members of $\mathcal{L}$ with probability above $1/2$, and in $\mathsf{BPTIME}[t]$ if moreover the error is at most $1/3$ on every input.

A language $\mathcal{L}$ is in $\oplus\mathsf{TIME}[t]$, $\mathsf{Mod}_q\mathsf{TIME}[t]$ or $\mathsf{C}_=\mathsf{TIME}[t]$ if a nondeterministic machine with running time $O(t)$, all of whose computations halt, has on input x a number of accepting computations that is odd, nonzero modulo q, or equal to its number of rejecting computations, respectively, exactly when $x \in \mathcal{L}$. The five classes just defined are the time-bounded versions of PP and BPP [Gil77], $\oplus$P [PZ83], Mod_qP [BG92] and $\mathsf{C}_=$P [Wag86].

Theorem 7.1. For every $t(n) \ge n$, $\mathsf{PPTIME}[t] \subseteq \mathsf{SPACE}[t/\sqrt{\log t}\,]$, and hence $\mathsf{BPTIME}[t] \subseteq \mathsf{SPACE}[t/\sqrt{\log t}\,]$. The same holds for $\mathsf{C}_=\mathsf{TIME}[t]$, $\oplus\mathsf{TIME}[t]$, and $\mathsf{Mod}_q\mathsf{TIME}[t]$ for fixed q. The simulating machines run in time $2^{O(t/\sqrt{\log t})}$.

Proof. Find, as in the proof of Theorem 2.4, the first clock u at which no computation is running at step $\bar{u}$. Let N_{ch} be the number of accepting choice strings of length $\bar{u}$, and N_{acc} and N_{rej} the numbers of accepting and rejecting computations. The rejecting computations of $\mathcal{M}$ are the accepting ones of $\mathcal{M}$ with its accepting and rejecting states exchanged, so by Lemma 2.2(c,d) each of the three counts is a sum, over the strings π of $|\pi| = O(u/\log u)$ bits, of network values, and each is at most 2^{2u}.

A bit of such a sum is computed from the bits of its summands one binary column at a time. At column i the simulator adds bit i of every summand to an incoming carry γ_i, outputs the parity, and passes on $\gamma_{i+1} = \lfloor (\sum_\pi \mathrm{bit}_i + \gamma_i)/2 \rfloor$. Since $\gamma_0 = 0$ and $\gamma_{i+1} \le (2^{|\pi|} + \gamma_i)/2$, every carry is at most $2^{|\pi|}$. So the current string, column and carry take $O(u/\log u)$ bits beyond the space of Theorem 2.3.

The tests themselves are short. For PPTIME, the machine tests $N_{\mathrm{ch}} > 2^{\bar{u}-1}$ by scanning the bits of N_{ch} from the top; this also handles BPTIME. For $\oplus\mathsf{TIME}$, it tests whether the lowest bit of N_{acc} is 1. For $\mathsf{Mod}_q\mathsf{TIME}$, it tests $N_{\mathrm{acc}} \not\equiv 0 \pmod q$, accumulating the residue from the highest of the $2u$ bits down with $O(\log q)$ bits of state. For $\mathsf{C}_=\mathsf{TIME}$, it compares N_{acc} and N_{rej} bit by bit. Everything fits in space $O(u/\sqrt{\log u})$, hence in time $2^{O(u/\sqrt{\log u})}$, and $u = O(t(|x|))$. □

Theorem 2.3 also gives a small-space version of the counting algorithms of [Ope26, Ant26].

Corollary 7.2. Given a 3-CNF formula with $m \ge 2$ clauses over variables numbered $1, \dots, n$, every one of which occurs, the number of its satisfying assignments can be computed in space $O(m/\sqrt{\log m})$ and time $2^{O(m/\sqrt{\log m})}$.

Proof. Put a vertex at every clause and a vertex at every occurrence of a variable. Join each occurrence to its clause, and join the occurrences of each variable in a path, in the order of the formula. An occurrence vertex requires the bits on all its edges to agree, and a clause vertex tests its clause, reading each literal, with its sign, from the edge to its occurrence. On each path a single truth value then fixes all the bits, so the satisfying assignments correspond one-to-one to the contributing edge assignments, because every variable occurs.

There are at most $3m$ edges to clauses and at most $3m$ edges on paths, every degree is at most 3, and every function is 0/1. Since $n \le 3m$, the formula has a description of length $O(m \log m)$, and the incidences and functions are found by scans with counters of $O(\log m)$ bits. So these networks have logspace access at size $s = 6m$, and Theorem 2.3 computes every bit of the count. □

8 Limits and open problems

Our simulation computes exact counts, so a stronger space bound would give a faster counting algorithm. By Corollary 7.2, for $D(s) \ge \log s$, a $D(s)$-space evaluator for networks with logspace access counts the satisfying assignments of 3-CNF formulas with m clauses in time $2^{O(D(O(m)))}$. Bounds $D(s) = s/\log s$ or $s^{1-\varepsilon}$ would therefore improve the known $2^{O(m/\sqrt{\log m})}$ counting time [Ope26, Ant26]. This limitation is specific to counting: Williams's simulation of deterministic time achieves a polynomial space saving without computing acceptance counts [Wil25]. Counting also places $\mathsf{PPTIME}[t]$ in the same space as $\mathsf{NTIME}[t]$ (Theorem 7.1).

We next examine the crossing decomposition itself. Replacing the signed sum by OR and AND would avoid cancellation, but would also force large factorization widths. As in [Ant26], a function Γ factors through e bits if $\Gamma(z, z') = \sum_{\theta \in \{0,1\}^e} \varphi(z, \theta)\varphi'(z', \theta)$, and over OR and AND if the same holds with $\bigvee$ and $\wedge$. Write r_i and c_i for the number of bits through which a piece Π_i of the crossing of two strings factors row-wise and column-wise. The following proposition bounds the widths of Boolean pieces directly in bits; we then compare this with the weight budgets of our signed pieces.

Proposition 8.1. Let $\Pi_1, \dots, \Pi_N \colon (\{0,1\}^\lambda)^4 \to \{0,1\}$ satisfy $\bigvee_i \Pi_i = [x = v][y = u]$, and let each Π_i factor over OR and AND through r_i bits row-wise and c_i bits column-wise. Then $r_i + c_i \geq 4\lambda - 2\log N$ for some i.

Proof. Each Π_i is at most the crossing, so in both directions it is a partial permutation matrix, and some Π_i has at least $2^{2\lambda}/N$ ones. A factorization through e bits covers the ones of Π_i by 2^e rectangles of ones of Π_i, and by the standard fooling-set argument [KN97] such a rectangle contains at most one nonzero entry of a partial permutation matrix. So 2^{r_i} and 2^{c_i} are at least $2^{2\lambda}/N$. □

So for $\log N = o(\lambda)$ some Boolean piece has $r_i^2 + c_i^2 \geq \frac{1}{2}(r_i + c_i)^2 \geq 8\lambda^2 - o(\lambda^2)$, four times the squared width $2\lambda^2$ of the crossing (the sum of the squared lengths of its two strings). Measured in weights rather than bits, the budgets of our pieces add up to twice the weight of the crossing. Over the integers, and for squared widths in bits, twice is the best possible when one diagonal is empty.

Proposition 8.2. Let $\Pi_1, \dots, \Pi_N \colon \{0,1\}^a \times \{0,1\}^a \to \mathbb{Z}$ satisfy $\sum_i \Pi_i(x, v) = [x = v]$, and let each Π_i factor through r_i bits and through c_i bits. Then $\min(r_i, c_i) \geq a - \log N$ for some i.

Proof. This is the crossing in which y and u are empty, so that both cuts separate x from v. The ranks of the Π_i over $\mathbb{Q}$ add up to at least the rank 2^a of the equality matrix, so some Π_i has rank at least $2^a/N$, and a function that factors through e bits has rank at most 2^e. □

For $\log N = o(a)$, some piece therefore passes nearly all a bits through each cut, and the sum of its squared widths is at least $2a^2 - o(a^2)$, twice the squared width of the crossing to leading order. If polynomially many pieces are to keep their squared widths within a fixed multiple of the squared width of the crossing for every shape of rectangle, that multiple must be at least 2.

The barrier depends on requiring cheap pieces for every rectangle shape. In Anthropic's equal-width recursion [Ant26], the condition is instead $r_i + c_i \leq 4\rho\lambda + O(\log \lambda)$. Restricting $y = u = 0^\lambda$ and applying Proposition 8.2 gives only $\rho \geq 1/2$, while the known pieces have $\rho = 1/\sqrt{2}$. The barrier therefore leaves room for better equal-width pieces and different recursions. Anthropic relates this question to compositions of quadratic forms [Hur98, Sha00] and partition rank [Nas20]. The barrier also leaves open methods tailored to blocked computations or methods that do not count.

Remark 8.3 (Relativization). In the sense of Baker, Gill and Solovay [BGS75], Theorem 1.1 and the separation of Corollary 1.2(i) do not relativize. Conventions for space-bounded oracle machines differ [LL76, RST84]. Give oracle machines a query tape that counts toward space, and let BSA be the set of strings $\langle \mathcal{M}' \rangle \# x \# 1^b$ such that $2^{|\langle \mathcal{M}' \rangle|} \leq b$ and the deterministic multitape machine $\mathcal{M}'$ accepts x in space b, in the sense that each of its configurations on x before it accepts, given by the state, the input head position in binary and the visited work cells, has a description of at most b bits. The padding lets $\langle \mathcal{M}' \rangle$ be decoded in space $O(b)$.

A machine with this oracle that uses space $s(n) \geq \log n$ asks queries of length at most $s(n)$, and it can answer each of them in space $O(s(n))$ without the oracle, so $\mathsf{SPACE}^{\mathsf{BSA}}[s] = \mathsf{SPACE}[s]$. One query decides any language in $\mathsf{SPACE}[n]$ in linear time, and trying the choice strings one at a time gives $\mathsf{NTIME}^A[n] \subseteq \mathsf{SPACE}^A[n]$ for every oracle A. Together,

$$\mathsf{TIME}^{\mathsf{BSA}}[n] = \mathsf{NTIME}^{\mathsf{BSA}}[n] = \mathsf{SPACE}^{\mathsf{BSA}}[n] = \mathsf{SPACE}[n].$$

If Theorem 1.1 held relative to BSA, then

$$\mathsf{SPACE}[n] \subseteq \mathsf{NTIME}^{\mathsf{BSA}}[n] \subseteq \mathsf{SPACE}^{\mathsf{BSA}}[n/\sqrt{\log n}] = \mathsf{SPACE}[n/\sqrt{\log n}],$$

contradicting the space hierarchy theorem. The same argument with $\mathsf{TIME}^{\mathsf{BSA}}[n]$ shows that the simulation of Hopcroft, Paul and Valiant does not relativize either. Both proofs use the locality of multitape computations, through blocking, whereas an oracle answer depends on a whole query string at once.

The most direct open problem is whether $\mathsf{NTIME}[t] \subseteq \mathsf{SPACE}[t/\log t]$. This would match Hopcroft, Paul and Valiant and answer their question, and even $t/(\log t)^{1/2+\delta}$ for some $\delta > 0$ would be new. Another is whether $\mathsf{NTIME}[t] \subseteq \mathsf{ATIME}[o(t)]$ for alternating machines with random access to the input, in analogy with $\mathsf{TIME}[t] \subseteq \mathsf{ATIME}[t/\log t]$ [DT85]. Our formula has depth $O(\log t)$, but its sums have index length up to about $t/\sqrt{\log t}$, and we do not know how to evaluate it in alternating time $o(t)$.

Acknowledgments

I thank my undergraduate advisor, Moshe Vardi, for introducing me to #SAT and for his help along the way. I also thank the organizers of the ETH Repository for collecting the work on refuting the Exponential Time Hypothesis.

AI methodology statement

The idea of the result is my own, and the work that followed was done jointly with Anthropic's Claude and OpenAI's ChatGPT. The models first helped me work through Anthropic's paper [Ant26]. I then proposed ideas that simplified the proofs and made the argument coherent, and the models contributed to exploring proof strategies, including the routing pieces of Section 5, to checking proofs, to the exposition and to the Lean formalization. I am responsible for all claims.

References

[AB09] S. Arora and B. Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.

[Ant26] Claude (Anthropic). A $2^{o(n)}$ time algorithm for 3-SAT. Manuscript, version of October 26, 2026, https://hexagonmath.org/pdf/2610.00577v1; Lean development at https://github.com/anthropics/formal-math/tree/main/eth.

[BG92] R. Beigel and J. Gill. Counting classes: thresholds, parity, mods, and fewness. *Theoret. Comput. Sci.*, 103(1):3–23, 1992.

[BGS75] T. Baker, J. Gill, and R. Solovay. Relativizations of the $\mathsf{P} =? \mathsf{NP}$ question. *SIAM J. Comput.*, 4(4):431–442, 1975.

[Bor77] A. Borodin. On relating time and space to size and depth. *SIAM J. Comput.*, 6(4):733–744, 1977.

[CLX09] J.-Y. Cai, P. Lu, and M. Xia. Holant problems and counting CSP. In *Proc. 41st ACM STOC*, pages 715–724, 2009.

[CM24] J. Cook and I. Mertz. Tree evaluation is in space $O(\log n \cdot \log\log n)$. In *Proc. 56th ACM STOC*, pages 1268–1278, 2024.

[Cmp] *Comparator*: checking Lean proofs against reference statements. Software, https://github.com/leanprover/comparator; merged into Lean core as `lake comparator` in v4.35.0.

[Coo88] S. A. Cook. Short propositional formulas represent nondeterministic computations. *Inform. Process. Lett.*, 26(5):269–270, 1988.

[Cplx] *Complexitylib*: computational complexity theory in Lean, built on CSLib. Software, revision a696d7b1 (now CSLib Frontier).

[CSL] *CSLib*: a Lean library for computer science. Software, https://github.com/leanprover/cslib.

[dMU21] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Proc. 28th CADE*, volume 12699 of *LNCS*, pages 625–635, 2021.

[DT85] P. W. Dymond and M. Tompa. Speedups of deterministic machines by synchronous parallel machines. *J. Comput. System Sci.*, 30(2):149–161, 1985.

[Eli75] P. Elias. Universal codeword sets and representations of the integers. *IEEE Trans. Inform. Theory*, 21(2):194–203, 1975.

[Gil77] J. Gill. Computational complexity of probabilistic Turing machines. *SIAM J. Comput.*, 6(4):675–695, 1977.

[GW83] H. Galperin and A. Wigderson. Succinct representations of graphs. *Inform. and Control*, 56(3):183–198, 1983.

[HPV75] J. Hopcroft, W. Paul, and L. Valiant. On time versus space and related problems. In *Proc. 16th IEEE FOCS*, pages 57–64, 1975.

[HPV77] J. Hopcroft, W. Paul, and L. Valiant. On time versus space. *J. ACM*, 24(2):332–337, 1977.

[Hur98] A. Hurwitz. Ueber die Composition der quadratischen Formen von beliebig vielen Variabeln. *Nachr. Ges. Wiss. Göttingen, Math.-Phys. Kl.*, pages 309–316, 1898.

[IP01] R. Impagliazzo and R. Paturi. On the complexity of k-SAT. *J. Comput. System Sci.*, 62(2):367–375, 2001.

[KN97] E. Kushilevitz and N. Nisan. *Communication Complexity*. Cambridge University Press, 1997.

[LL76] R. E. Ladner and N. A. Lynch. Relativization of questions about log space computability. *Math. Systems Theory*, 10:19–32, 1976.

[LN10] D. Lokshtanov and J. Nederlof. Saving space by algebraization. In *Proc. 42nd ACM STOC*, pages 321–330, 2010.

[Mat20] The mathlib Community. The Lean mathematical library. In *Proc. 9th ACM SIGPLAN CPP*, pages 367–381, 2020.

[McG26] S. McGuire. Non-trivial simulation of BPTIME. Note, October 30, 2026. ETH Repository.

[Nas20] E. Naslund. The partition rank of a tensor and k-right corners in $\mathbb{F}_q^n$. *J. Combin. Theory Ser. A*, 174:105190, 2020.

[Ope26] OpenAI. The Exponential Time Hypothesis is false. Manuscript, October 17, 2026; revised October 25, 2026. OpenAI Math Release.

[Pat72] M. S. Paterson. Tape bounds for time-bounded Turing machines. *J. Comput. System Sci.*, 6(2):116–124, 1972.

[PF79] N. Pippenger and M. J. Fischer. Relations among complexity measures. *J. ACM*, 26(2):361–381, 1979.

[PPST83] W. J. Paul, N. Pippenger, E. Szemerédi, and W. T. Trotter. On determinism versus non-determinism and related problems. In *Proc. 24th IEEE FOCS*, pages 429–438, 1983.

[PZ83] C. H. Papadimitriou and S. Zachos. Two remarks on the power of counting. In *Proc. 6th GI Conference on Theoretical Computer Science*, volume 145 of *LNCS*, pages 269–275, 1983.

[RST84] W. L. Ruzzo, J. Simon, and M. Tompa. Space-bounded hierarchies and probabilistic computations. *J. Comput. System Sci.*, 28(2):216–230, 1984.

[Ruz81] W. L. Ruzzo. On uniform circuit complexity. *J. Comput. System Sci.*, 22(3):365–383, 1981.

[San01] R. Santhanam. On separators, segregators and time versus space. In *Proc. 16th IEEE Conference on Computational Complexity*, pages 286–294, 2001.

[Sha00] D. B. Shapiro. *Compositions of Quadratic Forms*. de Gruyter, Berlin, 2000.

[SHL65] R. E. Stearns, J. Hartmanis, and P. M. Lewis II. Hierarchies of memory limited computations. In *Proc. 6th Annual Symposium on Switching Circuit Theory and Logical Design*, pages 179–190, 1965.

[SWS90] F. Schipp, W. R. Wade, and P. Simon. *Walsh Series: An Introduction to Dyadic Harmonic Analysis*. With the collaboration of J. Pál. Adam Hilger, Bristol, 1990.

[Val79] L. G. Valiant. Completeness classes in algebra. In *Proc. 11th ACM STOC*, pages 249–261, 1979.

[vM06] D. van Melkebeek. A survey of lower bounds for satisfiability and related problems. *Found. Trends Theor. Comput. Sci.*, 2(3):197–303, 2006.

[Wag86] K. W. Wagner. The complexity of combinatorial problems with succinct input representation. *Acta Inform.*, 23(3):325–356, 1986.

[Wil05] R. Williams. A new algorithm for optimal 2-constraint satisfaction and its implications. *Theoret. Comput. Sci.*, 348(2–3):357–365, 2005.

[Wil25] R. Williams. Simulating time with square-root space. In *Proc. 57th ACM STOC*, pages 13–23, 2025.

[Zhu26] S. Zhuang. Uncrossing wires for 3-SAT: a potential that closes the recursion. Manuscript, October 31, 2026. https://github.com/suquark/NotETHSimplified.

A Deferred proofs

Proof of Lemma 4.3. Put $K = q_*/2$. The nodes $\{0,1\}^{h_*}$ are split by their first bit into K vertex groups and K test groups. List the vertices of G in the order of their names, let $\mathrm{pre}(v)$ be the number of ports of the vertices before v, put $D_0 = \max\{1, \lceil 2M/K \rceil\}$, and put v into vertex group $\min\{K-1, \lfloor \mathrm{pre}(v)/D_0 \rfloor\}$. Vertex group $i < K-1$ consists of vertices with $iD_0 \le \mathrm{pre}(v) < (i+1)D_0$, so its number of ports d_i is at most $D_0 + \Delta$, since its last vertex has at most Δ ports. If the last group is nonempty, it starts at a vertex with $\mathrm{pre}(v) \ge (K-1)D_0$, so $d_{K-1} \le 2M - (K-1)D_0 \le D_0$. Isolated vertices are placed in the same way and have no ports.

Now subdivide every edge e of G by a new vertex with the function $[z_1 = z_2]$ of its two ports. The bit of e becomes two bits constrained to agree, so every assignment of G has exactly one consistent extension, and the value does not change. For each unordered pair $\{i, j\}$ of vertex groups, including $i = j$, list the edges between groups i and j in the order of their names, and put the test of the edge in position $\nu \ge 0$ of this list into test group $\nu \bmod K$. Each node of

$\mathcal{C}_0$ is now a set of vertices of the subdivided network, every edge of which joins a vertex group to a test group. Let the width b of the pair (vertex group i, test group τ) be the number of half-edges between them, listed in the order of edge names and, for an edge inside group i, of its two endpoints; give this pair the budget $b(b+1)/2$, and every other pair the budget 0. A vertex-group table is 0 unless each of its labels is $\mathrm{enc}(\theta)$ for a string θ of the width of its pair, and otherwise it is the product of the functions f_v of its vertices, whose arguments are read from these strings. A test-group table is likewise 0 unless its labels encode strings of the right widths, and otherwise the product of its equality tests. Neither has a sum: no subdivided edge lies inside a group. Since enc is injective and the two copies of each label agree, $Z(\mathcal{C}_0) = Z(G)$. This proves (b).

For (a), let m_{ij} be the number of edges between vertex groups i and j. The width between vertex group i and a test group is at most

$$\sum_{j\neq i}\left\lceil\frac{m_{ij}}{K}\right\rceil + 2\left\lceil\frac{m_{ii}}{K}\right\rceil \le \frac{d_i}{K} + K + 1 \le \frac{2M}{K^2} + \frac{\Delta+1}{K} + K + 2 =: w_{\max},$$

since $\sum_{j\neq i} m_{ij} + 2m_{ii} = d_i$. At most K^2 budgets are nonzero, each at most $w_{\max}^2$, so $\Phi(\mathcal{C}_0) \le (q_*-1)^2K^2w_{\max}^2 \le (2K^2w_{\max})^2 = O((M + q_*\Delta + q_*^3)^2)$, which is $O(s^2)$ because $q_*^3 \le s$.

For (c), $\mathrm{pre}(v)$, the group of a vertex, the position of an edge in its list, and the number of half-edges before a given one on a pair are counts over the vertices and edges of G, each computed by a constant number of nested scans with counters of $O(\log s)$ bits. Parallel edges and edges inside a group are handled by the same counts. By Lemma 4.1, the machine checks from the code whether a label is $\mathrm{enc}(\theta)$ for a string of b bits and, if so, reads θ_ν. □

Proof of Lemma 5.6. We treat $N^{\uparrow}(x, y; m)$; the factor $N^{\downarrow}(u, v; m)$ is handled by exchanging η and ζ and using the lower rules. A source is a request of out_x, out_y or ζ, and a target a request of in_x, in_y or η; each has a kind (x, y or the message) and a type. List the sources by kind, then type, then occurrence. For a source s and a target t let $\omega(s,t)$ be the rule of an upper half-routing: R_{xy} between kinds x and y, the step $(1-z^2)C_{i,n}$ between kind x and the message, the copy $[n=j]$ between kind y and the message, and 0 for two requests of the same kind. Then $N^{\uparrow}(x, y; m) = \sum_\rho \prod_s \omega(s, \rho(s))$, over the maps ρ from sources to targets that send to every kind and type of target exactly its multiplicity. Every coefficient of $\omega(s,t)$ is in $\{-1, 0, 1\}$ and can be computed in space $O(\log(W+2))$, and every exponent of $\omega(s,t)$ is at most $\mathrm{wt}\, s + \mathrm{wt}\, t$ in absolute value, by (2), since $d(i,j) + 2 \le (i+1) + (j+1)$.

For an interval I of the list of sources and a multiset $\mathcal{T}$ of targets, let $N(I, \mathcal{T})$ be the same sum restricted to them, and $W(I, \mathcal{T})$ the total weight of I and $\mathcal{T}$; so every exponent of $N(I, \mathcal{T})$ is at most $W(I, \mathcal{T})$ in absolute value. Given a map ρ for $(I, \mathcal{T})$, go through the sources of I in order, adding up the weight of each source and of its target, and let s be the first source at which this running sum reaches $W(I, \mathcal{T})/2$. The sources before s form an interval I_1, sent to a multiset $\mathcal{T}_1$ of targets, and those after it form an interval I_2, sent to $\mathcal{T}_2 = \mathcal{T} - \mathcal{T}_1 - t$, where $t = \rho(s)$. By the choice of s, $W(I_1, \mathcal{T}_1) < W(I, \mathcal{T})/2$ and $W(I_2, \mathcal{T}_2) \le W(I, \mathcal{T})/2$. Conversely, maps for $(I_1, \mathcal{T}_1)$ and $(I_2, \mathcal{T}_2)$ and a target t for s combine into exactly one map for $(I, \mathcal{T})$, in which s is that first source exactly when

$$W(I_1, \mathcal{T}_1) < W(I, \mathcal{T})/2 \le W(I_1, \mathcal{T}_1) + \mathrm{wt}\, s + \mathrm{wt}\, t. \qquad (*)$$

Since targets of the same kind and type are interchangeable, t ranges over kinds and types and

no multiplicity factor arises. Hence

$$N(I,\mathcal{T}) = \sum_{s,t,\mathcal{T}_1} [(*)]\; N(I_1,\mathcal{T}_1)\,\omega(s,t)\,N(I_2,\mathcal{T}_2),$$

with $N(\emptyset,\mathcal{T}) = [\mathcal{T}=\emptyset]$. Taking the coefficient of z^c splits c into the exponents $c_1, c_0, c-c_1-c_0$ of the three factors, of which the first two are at most $W(I,\mathcal{T})$ in absolute value.

This is a formula. At a subproblem $(I,\mathcal{T},c)$ of weight W' there is a sum node over $(s,t,\mathcal{T}_1,c_1,c_0)$ above a product node with index length 2, whose children are the two subproblems, the leaf $[z^{c_0}]\omega(s,t)$ and the leaf 1. Index tuples that violate a condition lead to the leaf 0, a subproblem with $|c|>W'$ is the leaf 0, and a subproblem with $I=\emptyset$ is the leaf $[\mathcal{T}=\emptyset][c=0]$. The sum index has fields of $O(\log(W+2))$ bits for s, t, c_1 and c_0 and, by Lemma 4.1, a field of $w(W')$ bits for the targets of each kind in $\mathcal{T}_1$, so it takes $O(\sqrt{W'}+\log(W+2))$ bits. The weight at least halves at every level, and a nonempty subproblem has weight at least 1, so the recursion has at most $\log(W+1)+2$ levels, each a sum node above a product node of index length 2, and the ℓ-cost is at most

$$\sum_{d=0}^{\lfloor\log(W+1)\rfloor+1} O\big(\sqrt{W/2^d}+\log(W+2)+\ell\big) = O\big(\sqrt{W}+(\ell+\log(W+2))\log(W+2)\big).$$

An explicit machine follows the address from the root, keeping only the current interval and the current multiset of targets: entering the first subproblem, it reads $\mathcal{T}_1$ from the address, and entering the second, it writes the code of $\mathcal{T}_2=\mathcal{T}-\mathcal{T}_1-t$ into a second buffer and then replaces the code of $\mathcal{T}$ by it. It finds the kind and type of any source by scanning the codes of x, y and m, checks $(*)$ by computing weights, and evaluates the leaves with the rules. A constant number of codes of $O(\sqrt{W}+1)$ bits and counters of $O(\log(W+2))$ bits suffice. □

Proof of Lemma 6.4. Given an address, the machine follows it from the root. While following clique values, it keeps the budget array of every clique it passes. Each array has at most q_*^2 numbers of $O(\log s)$ bits, for $O(q_*^2 \log s \log h_*)$ bits in all. When it reaches table entries, it uses these arrays again, since an entry refers to the clique from which its table was formed. The budgets of $\mathcal{C}_0$ are computed by Lemma 4.3(c). At a sum node over the choices, the machine reads the grade and split of each rectangle from the address, checks that the grade is even and that both are in range, forms the two candidate budget arrays a^{row} and a^{col} from the grades, compares the two potentials, and moves to the cheaper array. The index length of every node follows from the budget arrays: the length of a choice is the number of rectangles times the fixed field length, the length of ξ is the sum of $w(a)$ over the copies in one row or column, and the length at name length 1 is $w(a)$ for the single budget a. These are computed by summations with $O(\log s)$-bit counters.

The argument of an old table entry is a slice of the current ξ, and the offsets of the codes inside ξ are prefix sums of the values $w(a)$. Validity of a packed label $\tilde{\lambda}$ is checked by scanning its code. A label λ or a message m_R inside $\tilde{\lambda}$ is not written out. The machine reads it on demand by scanning the code of $\tilde{\lambda}$, which lists its distinct weights in increasing order, with multiplicities. To recover stream ν in one direction, it keeps the packed types congruent to ν modulo L and inverts the map $i \mapsto Li+\nu$ of (6), preserving their order. It then computes any bit of the code of λ or m_R by regenerating the gaps and multiplicities, in space $O(\log s)$. An equality test compares two such lists entry by entry. A weight test uses Lemma 4.1. A leaf of $\mathcal{C}_0$ is evaluated in space $O(\log s)$ by Lemma 4.3(c). Inside the recursion for a coefficient of $N^{\uparrow}$ or $N^{\downarrow}$, the machine runs

the explicit machine of Lemma 5.6, simulating its read access to the codes of the views and the message in the same way, in space $O(\sqrt{W} + \log^2 s) = O(F)$, by the bound on W in the proof of Lemma 6.3. Every other code it scans or writes has $O(F)$ bits too, because by that lemma every budget is at most $\Phi/(q-1)^2 \le \Phi_*/h_*$. With $q_*^2 \le s^{2/3}$ for the budget arrays, the total workspace is $O(F)$. All subroutines halt. □

Proof of Corollary 1.2(iii). Let $\mathcal{L} = \mathcal{L}_S$ for $S(n) = n$, let $\mathcal{M}$ be a nondeterministic machine deciding $\mathcal{L}$ or its complement, with running time $T_{\mathcal{M}}(n)$, and put $T^*(n) = \max(T_{\mathcal{M}}(n), n)$. The proof of Theorem 2.4, with Theorem 2.3 and flipping the answer if necessary, gives a deterministic machine deciding $\mathcal{L}$ with at most $a_1 T^*(n)/\sqrt{\log T^*(n)}$ work cells on inputs of length n, for a constant a_1 and all large n. By Lemma 2.5 there is $a_2 > 0$ with $a_1 T^*(n)/\sqrt{\log T^*(n)} > a_2 n$ for all large n. Since $T^*(n) \ge n$, this gives $T^*(n) > (a_2/a_1)\, n\sqrt{\log n}$, which exceeds n for large n, so $T^*(n) = T_{\mathcal{M}}(n)$. □

B The almost-everywhere space hierarchy theorem

We recall the standard proof. Encode deterministic machines by binary strings $\langle \mathcal{M} \rangle$ that list the states, so that $\mathcal{M}$ has at most $|\langle \mathcal{M} \rangle|$ states. On an input w of length n, the machine $\mathcal{D}$ rejects unless $w = \langle \mathcal{M} \rangle 10^j$ for a machine $\mathcal{M}$ and some $j \ge 0$. It marks $S(n)$ cells on each work tape and $C_0 S(n)$ cells on a counter tape, where $C_0 = 3 + \lceil \log |\Sigma| \rceil$ for the work alphabet Σ of $\mathcal{D}$, rejects if $|\langle \mathcal{M} \rangle| > S(n)$, and otherwise simulates $\mathcal{M}$ on w. It keeps the state of $\mathcal{M}$ on a work tape and keeps the work tapes of $\mathcal{M}$, with head marks, on tracks of one work tape, using $c_{\mathcal{M}}$ cells per cell of $\mathcal{M}$, where $c_{\mathcal{M}}$ depends only on the tapes and alphabet of $\mathcal{M}$; it copies $\langle \mathcal{M} \rangle$ to a work tape, using at most $S(n)$ cells, and its own input head serves $\mathcal{M}$. $\mathcal{D}$ rejects if the stored tapes would leave the marked cells or the step counter overflows; if the simulation halts, $\mathcal{D}$ accepts exactly when $\mathcal{M}$ rejects. So $\mathcal{L}_S = \mathcal{L}(\mathcal{D}) \in \mathsf{SPACE}[S]$.

While the stored tapes fit, a configuration of $\mathcal{M}$ on w is determined by its state, its input head position and the marked cells, head marks included; for $n > 4$ there are at most $|\langle \mathcal{M} \rangle|(n+2)|\Sigma|^{S(n)} \le 2^{S(n)} \cdot 2^{S(n)+2} \cdot |\Sigma|^{S(n)} < 2^{C_0 S(n)}$ of them, since $|\langle \mathcal{M} \rangle| \le S(n)$ and $\log n \le S(n)$.

Now let $\mathcal{M}$ decide $\mathcal{L}_S$, let $n > \max(|\langle \mathcal{M} \rangle|, 4)$, and suppose $\mathcal{M}$ visits at most $S(n)/c_{\mathcal{M}}$ work cells on all inputs of length n. On $w = \langle \mathcal{M} \rangle 10^{n-|\langle \mathcal{M} \rangle|-1}$ the stored tapes fit, and the simulation ends before the counter overflows, because $\mathcal{M}$ halts and a halting deterministic computation never repeats a configuration. Then $\mathcal{D}$ answers the opposite of $\mathcal{M}$ on w, a contradiction, so the lemma holds with $c = 1/c_{\mathcal{M}}$.